

第 2 章 构建智能体

【本章学习目标】

本章承接第 1 章 LangChain 框架六大核心组件基础，聚焦智能体开发核心底层逻辑，补齐前期遗漏的提示词核心原理、消息角色分层、人机交互规则等关键知识点，打通基础模型调用—可控提示词工程—结构化消息交互—智能体自主决策的完整技术链路。学完本章可达成核心能力：

- 深度掌握提示词的核心作用、核心构成与工程价值，精准区分 System 系统角色、Human 用户角色、AI 助手角色的本质差异与使用场景
- 厘清普通 LLM 被动调用与 AI 智能体主动执行的核心区别，吃透智能体感知-思考-行动-观察闭环运行逻辑与四大核心特征
- 精通 LangChain 智能体五大核心组件协同原理，掌握结构化消息、提示词模板、模型初始化、自定义工具、长短记忆的全套实操技术
- 通过全新原创案例落地智能体全链路开发，掌握自主工具调用、多轮上下文记忆、结构化输出的工程化实现方案
- 规避提示词编写、角色配置、工具开发、记忆使用的高频易错点，统一全书智能体开发技术口径与规范

本章为课程核心基础章节，聚焦 AI 智能体核心原理与基础搭建实操，彻底区分传统 LLM 与智能体的能力差异，循序渐进讲解智能体的定义、特征、架构、核心组件及基础落地案例，适配零基础开发者入门智能体开发。核心围绕智能体感知-思考-行动-观察闭环逻辑展开，打破传统大模型“仅文本生成、无法自主执行任务”的局限，讲解智能体从被动应答到主动决策、工具调用、任务执行的能力升级逻辑。

2.1 智能体核心认知与体系架构

2.1.1 智能体的核心定义与核心价值

智能体（Agent，也常称为 AI Agent）是人工智能领域中能够感知环境、自主决策、执行动作并优化结果以实现预设目标的自主系统。它突破了传统大型语言模型（LLM）“只说不做”的局限，通过“感知-思考-行动-观察”的闭环，将 AI 能力从“文本生成”升级为“任务执行”，成为连接 AI 与现实世界的核心载体。

原生 LLM 仅能依托训练数据集完成被动文本输出，不存在自主思考、外部操作的能力，天然存在知识滞后、幻觉编造的缺陷；基础 Chain 链路需要开发者提前写死全部执行流程，无论用户提问如何变化，都只能机械执行预设步骤，无法适配模糊、多目标的复杂需求。而 LangChain 智能体依托 ReAct 推理行动机制，完整实现“思考决策—工具执行—结果观察—迭代优化”的循环流程，能够自主拆解任务、筛选外部资源，从根源弥补原生大模型与固定链路的各项短板，实现真正的自主智能交互。

一句话总结：AI 智能体能够自主感知外部环境、解析用户自然语言目标、自主推理执行逻辑、主动调用外部工具、接收环境反馈迭代、输出标准化合规结果的智能化执行系统。

2.1.2 智能体的四大核心特征

完整工业级可投入使用用的 LangChain 智能体必须同时具备四大核心特征，这也是区分普通 LLM 问答、固定脚本与标准智能体的判定依据，四项缺一不可：

（1）自主性（Autonomy）：无需人工分步干预任务流程，智能体可基于用户目标与当前环境信息自主判断“需要做什么、以何种顺序完成”，自主筛选适配工具、拆分子任务、规划完整执行路径，全程不需要开发者预设全套执行脚本，独立走完完整任务闭环。

（2）交互性（Interactivity）：支持双向动态交互能力，智能体可同时对接用户输入、本地文档、第三方网络 API、内存存储等多类内外资源，接收工具、用户的反馈信息后，动态调整后续推理与执行策略，不会机械重复固定操作。

（3）目标导向性（Goal-Oriented）：智能体全部思考、执行动作都围绕用户核心需求展开，内置复杂任务拆解逻辑，能够自动区分有效步骤与无效冗余操作，对子任务划分优先级，优先完成贴合用户目标的执行流程，杜绝无意义循环调用。

（4）适应性（Adaptability）：依托内置记忆系统留存全部交互经验，能够适配残缺提问、模糊口语、场景变更等各类不确定输入，不会单一、机械地套用同一套应答逻辑，可根据对话上下文灵活调整推理思路。

2.1.3 智能体主流分类与适配场景

根据智能体的记忆能力、决策逻辑、应用复杂度，行业将智能体划分为六大主流类型，结合本课程智能搜索开发主线，课堂实训以目标导向智能体、学习型智能体为核心讲解对象，其余类型作为拓展理论补充。

（1）反应式智能体（Reactive Agent）：无记忆模块，仅依据当前单次输入即时给出响应，逻辑极简、响应速度快，适配工业设备监控、固定单指令应答等轻量化场景，不存在多轮对话交互能力。

（2）基于模型的智能体（Model-Based Agent）：内置环境状态模拟逻辑，可预判工具调用、外部接口返回的结果变化，适配输入动态变化的交互场景，能够根据预判调整工具调用顺序。

(3) 目标导向智能体 (Goal-Based Agent)：以用户明确任务目标为核心，原生支持复杂任务分层拆解，是商用智能问答、私有知识库检索系统的主流实现类型，本课程天气、知识库工具案例均基于该类型开发。

(4) 效用函数智能体 (Utility-Based Agent)：引入成本、优先级、容错率等量化评估规则，可在多条可行执行路径中筛选最优方案，适配资源分配、多维度方案对比等优化类场景。

(5) 学习型智能体 (Learning Agent)：依托长期记忆存储历史交互数据，每一轮对话后自动优化自身决策逻辑，交互次数越多应答精准度越高，适配长期个性化交互项目。

(6) 多智能体系统 (Multi-Agent System)：由多个功能差异化智能体分工协作，分别负责检索、计算、文案生成等细分模块，适配智慧城市、大型数据分析等超复杂工程项目。

2.1.4 智能体完整架构与运行闭环

LangChain 体系下所有标准智能体统一遵循五层分层架构，五层模块层层联动、数据双向流转，共同支撑“感知 - 记忆 - 推理 - 规划 - 执行”的完整自主运行闭环，各层级权责完全解耦，可单独替换、灵活组合。

(1) 感知层：核心功能为全量信息采集与标准化封装，统一接收用户自然语言提问、工具返回原始数据、历史对话记录等全部输入内容，将各类零散信息转换为 LangChain 标准化消息对象，向下传递至记忆层与推理层，是智能体所有输入信息的统一入口。

(2) 记忆层：负责全部交互数据的存储与调取，分为短期会话内存记忆、Redis 长期持久化记忆两类，完整留存对话上下文、工具执行中间结果，保障多轮连续提问时，智能可读取历史内容连贯作答，是智能体适应性特征的底层支撑。

(3) 推理层：智能体的核心决策大脑，载体为 LLM 大模型，依托 ReAct 机制完成语义理解、需求拆解、工具匹配、参数生成全部推理工作，接收感知层、记忆层传递的消息后，输出下一步行动指令。

(4) 规划层：负责复杂任务调度，接收推理层的核心目标后，将单一复杂需求拆分为有序子任务，划分各子任务执行优先级，避免无序重复调用工具，简化推理层的决策负担。

(5) 执行层：承载所有外部操作，由各类自定义 Tool 工具集合组成，接收规划层下发的行动指令，完成网络请求、文档读取、数值计算等具象操作，执行完成后将结果封装为工具消息，反向回流至推理层，形成迭代闭环。

智能体标准化完整工作闭环完整流转流程：用户输入自然语言请求进入感知层 — 感知层将内容封装为标准化消息 — 记忆层读取当前会话全部历史消息补充上下文 — 推理层结合系统提示、历史消息完成自主推理，判断是否需要调用工具、调用哪一款工具、传入对应参数 — 执行层运行指定工具获取外部原始数据，生成工具消息 — 工具消息回流至推理层，模型整合全部消息二次推理，判断信息是否充足 — 信息充足则生成标准化应答，存入记忆层并对外输出；信息不足则再次发起工具调用循环。全程无需人工介入，完整实现自主闭环执行。

2.1.5 LangChain 智能体核心组件

LangChain 官方将智能体五层架构工程化为五大解耦核心组件，五大组件可自由搭配组合，配套实训可按需删减模块，组件功能、实操示例统一整理为表 2.1-1：

表 2.1-1 Agent 核心 组件

组件	作用	示例
Agent (智能体)	全局任务决策者与调度者，统筹全流程，解析用户目标、推理执行逻辑、选择匹配工具、规划执行流程	自主判断知识类提问检索知识库、实时信息提问调用联网工具
LLM (大模型)	核心推理引擎，提供语义理解、逻辑推理、文本生成、结构化输出能力，支撑 Agent 所有决策逻辑	硅基流动 Qwen3-8B 模型，解析指令、生成工具调用参数、整合输出答案
Tools (工具集)	能力拓展执行器，弥补大模型知识滞后、无实操能力的短板，对接外部资源、完成具象化操作	时间查询、知识库检索、文本解析、联网搜索工具
Memory (记忆系统)	上下文存储载体，留存对话历史、任务中间数据，保障多轮交互连贯、任务可迭代优化	内存短期会话记忆、Redis 长期持久化记忆
OutputParser (输出解析器)	规整模型自由文本输出，转化为标准化结构化数据，适配项目对接、前端展示、接口返回场景	JSON 标准化输出、自定义字段解析、格式纠错

LangChain 智能体标准循环工作流程图 2.1-1 所示：

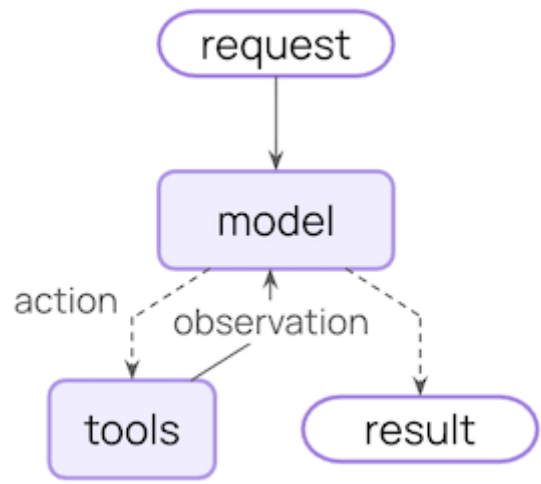


图 2.1-1 Agent 工作流程

工作流程拆解说明：

- (1) 输入请求 (request)：用户原始提问作为初始输入，统一封装为 HumanMessage 消息传入模型；
- (2) 模型决策 (model)：LLM 读取系统消息、历史对话消息完成推理，存在两条分支：若无需工具，直接生成最终 result 应答；若需要工具，则输出 action 工具调用指令；
- (3) 工具执行与反馈 (tools/observation)：工具接收 action 指令运行，将返回数据封装为 ToolMessage 作为 observation 观察信息，重新传回模型再次推理；
- (4) 结果输出 (result)：模型整合全部消息后生成标准化应答，存入记忆并对外展示。

2.1.6 快速实战案例

全书实操案统一适配课程标准化开发环境，统一镜像源、模型、安全配置，规避环境报错、接口异常问题，全程技术口径统一：

1、依赖库安装

项目依赖统一安装命令，全程使用清华镜像源加速下载，解决超时、安装失败问题

```
# 激活环境
conda activate ai_env
# 安装依赖
pip install python-dotenv langgraph-checkpoint redis -i
https://pypi.tuna.tsinghua.edu.cn/simple
```

2、环境配置规范

项目根目录新建.env 全局配置文件隔离敏感信息，企业级安全规范，密钥禁止代码硬编码。具体配置步骤如下：

- (1) 新建专属项目文件夹，示例路径：E:\user\ai\langchain_exam；
- (2) 使用开发工具打开项目文件夹，在项目根目录创建.env 全局配置文件；
- (3) 在文件中写入硅基流动平台（或其它平台）标准接口配置，统一全局接口地址与密钥认证：

```
OPENAI_API_KEY=你的硅基流动 API 密钥
BASE_URL=https://api.siliconflow.cn/v1
CHAT_MODEL=Qwen/Qwen3-8B
```

3、模型统一标准

全书实操案例默认使用硅基流动开源模型：Qwen/Qwen3-8B，通过 temperature 参数精准适配不同业务场景，参数统一规范：精准问答、技术答疑、工具调用场景设置 0.1-0.3，保障输出严谨无随机偏差；创意文案、拓展生成场景设置 0.7-0.9，保留内容创作灵活性。

4、例题调试规范

全书实操案例统一使用 Jupyter Notebook 分步调试、运行与效果验证，如，创建第二章.ipynb 文件，代码支持分段运行、实时查看结果、逐行打印消息排查逻辑，适配课堂分步演示、学生自主排错的授课需求。

全书通用标准化模型初始化基础模板，所有案例统一复用，减少重复代码编写：

```
# 1. 导入核心依赖库
import os
from langchain_openai import ChatOpenAI
from langchain.agents import create_agent
from dotenv import load_dotenv

# 2. 加载环境变量（企业级安全规范，覆盖重复配置）
load_dotenv(override=True)

# 3. 初始化标准化模型（全书统一配置）
llm= ChatOpenAI(
    base_url=os.getenv("BASE_URL"),
    openai_api_key=os.getenv("OPENAI_API_KEY"),
```

```
model=os.getenv("CHAT_MODEL"),
temperature=0.1 # 低温度适配精准问答场景，规避随机输出偏差
)
```

【例 2.1-1】基础角色定制智能体

本案例聚焦系统提示词控参核心逻辑，通过定制系统角色约束模型行为，让初学者直观理解提示词定义能力、模型执行能力的基础架构，无复杂工具依赖，快速入门可控 AI 开发。

【程序代码】

```
# 1. 系统角色定义（核心：约束模型身份、能力、输出规则）
SYSTEM_PROMPT = """你是专业的 Python 编程助教。

【强制硬性规则，绝对不可违反】

1. 只允许回答 Python 编程、代码语法、Python 实操相关问题
2. 所有非 Python 相关问题，必须直接固定回复：抱歉，我仅可解答 Python 基础编程问题，不支持该类需求。
3. 禁止强行作答、禁止跨界创作、禁止自作主张回答无关内容
"""

# 2. 构建专属智能体
agent = create_agent(model=llm, system_prompt=SYSTEM_PROMPT)

# 3. 场景测试
if __name__ == "__main__":
    # 合法场景提问
    res1 = agent.invoke({"messages":["请写一个 Python 读取本地 txt 文件的极简代码"]})
    print("编程助教应答：\n", res1["messages"][-1].content)
    # 跨界提问（验证角色约束效果）
    res2 = agent.invoke({"messages":["帮我写一篇古代人物的作文"]})
    print("\n 跨界问题应答：\n", res2["messages"][-1].content)
```

【运行结果解读】

程序执行后分为两段输出，第一段为合规代码提问应答，模型严格按照系统提示输出带单行注释的极简文件读取代码；第二段用户提出作文需求，用户输入优先级低于系统全局规则，模型直接输出固定兜底回复，不会生成作文内容。本案例完整验证消息层级差异、提示词强制约束两大核心知识点，是后续所有复杂智能体的交互底层基础。

【例 2.1-2】多功能天气查询智能体

本案例串联工具开发、外部 API 调用、结构化输出、会话记忆四大智能体核心技术，衔接【例 2.1-1】基础角色约束案例，由单一角色管控升级为带外部资源交互的工程化智能体，是智能搜索项目工具调用模块前置核心实训。

运行后智能体完整实现 6 项核心能力：

- ①自主判断调用天气工具，不会编造气象数据；
- ②读取 API 接口获取实时天气原始数据；
- ③按照固定 Weather 模型结构化输出，字段统一规范；

- ④依托 `thread_id` 保存对话上下文，支持无城市追问；
- ⑤接口异常自动捕获，返回友好错误提示；
- ⑥全程遵循系统提示词约束，不会生成虚假天气信息。

【程序代码】

```
# 0. 引入依赖包
import requests
from pydantic import BaseModel, Field
from langchain.tools import tool
from langgraph.checkpoint.memory import InMemorySaver

# 1. 精细化系统提示词（清晰工具说明+行为约束，规范智能体决策逻辑）
SYSTEM_PROMPT = """你是专业天气预报助手，仅可调用 get_weather_for_location 工具
查询城市天气，严格遵循规则：
    1. 用户询问任意城市天气，必须调用工具获取实时数据，禁止编造气温、天气状况；
    2. 工具返回结果后，按照指定结构化格式整理输出；
    3. 无对应城市数据如实告知，禁止虚构天气信息；
    4. 可结合上下文记忆回答用户追问。
    可用工具：get_weather_for_location（输入中文城市名，返回近 3 天分段天气）
    """

# 2. 创建外部天气查询工具（对接公共天气接口）
@tool
def get_weather_for_location(city: str) -> str:
    """
    根据中文城市名称直接查询当日天气预报
    参数 city: 中文城市名称，例如南昌、北京、上海
    返回：当日天气
    """
    url = f"https://cn.api.hz.cn/api/tianqi/tqyb.php?id=10019433&key=fe034a161aee2bf45f841b6024e898b&place={city}"

    try:
        res = requests.get(url, timeout=10)
        res.raise_for_status()
        data = res.json()
        # 接口返回非 200 代表查询失败
        if data.get("code") != 200:
            return f"天气查询失败：{data.get('msg')}，请确认城市名称正确"

        return {"weather": data['weather1'], "temperature": data['wd1']}

    except Exception as err:
```

```

        return f"气象接口请求异常，查询失败：{str(err)}"

# 3. Pydantic 强类型结构化输出定义（固定返回字段，结果可直接对接接口）
class WeatherOutput(BaseModel):
    city: str = Field(description="查询目标城市名称")
    temperature: str = Field(description="当日温度")
    weather: str = Field(description="天气状况，如多云、阵雨")

# 4. 初始化内存记忆组件，支持同会话上下文追问
checkpointer = InMemorySaver()

# 5. 组装完整智能体：模型+工具+提示词+结构化输出+记忆
agent = create_agent(
    model=llm,
    system_prompt=SYSTEM_PROMPT,
    tools=[get_weather_for_location], # 工具集
    response_format=WeatherOutput, # 结构化输出
    checkpointer=checkpointer # 记忆组件
)

# 7. 多轮对话测试（统一 thread_id 绑定会话记忆）
if __name__ == "__main__":
    # 唯一会话标识，上下文共享
    chat_config = {"configurable": {"thread_id": "weather_chat_001"}}
    # 第一轮：查询南昌天气
    res1 = agent.invoke(
        {"messages": [{"role": "user", "content": "今天南昌的天气如何？"}]},
        config=chat_config
    )
    print("第一轮天气查询结构化结果：")
    print(res1["structured_response"])
    print("-" * 60)
    # 第二轮上下文追问，无需重复输入城市
    res2 = agent.invoke(
        {"messages": [{"role": "user", "content": "再打印一遍这座城市的天气情况"}]},
        config=chat_config
    )
    print("上下文追问结构化结果：")
    print(res2["structured_response"])

```

【运行结果解读】

第一轮调用执行全流程拆解

①用户输入：今天南昌的天气如何？存入用户消息；

②智能体读取系统提示，识别需求为城市天气查询，判定需要执行

get_weather_for_location 工具；

- ③自动提取参数 city="南昌"，发起 wttr.in 公共天气接口网络请求；
- ④接口返回南昌近 3 天分时段原始天气文本，智能体清洗温度、天气关键词；
- ⑤按照预先定义的 WeatherOutput Pydantic 模型，标准化封装城市、温度、天气三类字

段；

- ⑥输出结构化对象，数据规范、可直接对接前端 / 数据库接口。

第二轮上下文追问执行核心亮点

用户仅输入再打印一遍这座城市的天气，未重复填写城市名称，但智能体成功输出南昌天气：

核心原理：代码中统一 thread_id="weather_chat_001"会话标识，InMemorySaver 内存记忆组件完整留存本轮对话上下文；

智能体读取历史对话记录，自动复用“南昌”作为查询参数，无需用户重复输入，实现类人类连续聊天交互效果。

例如，再执行下面代码：

```
# 8. 再次执行代码（统一 thread_id 绑定会话记忆）
if __name__ == "__main__":
    # 唯一会话标识，上下文共享
    chat_config = {"configurable": {"thread_id": "weather_chat_001"}}
    res3 = agent.invoke(
        {"messages": [{"role": "user", "content": "我刚才询问的是哪个城市的天气？"}]},
        config=chat_config
    )
    print("上下文追问结构化结果：")
    print(res2["structured_response"])
```

会把刚才查询过的城市的天气再打印一次。

本案例完整覆盖 LangChain 智能体五大核心组件，是【例 2.1-1】纯角色约束基础智能体的进阶拓展：前者仅依托 LLM + 简单 Agent 实现无工具纯文本问答管控，缺失工具、记忆、结构化输出三大组件；本案例完整落地 LLM 推理引擎 + 外部 Tools 实时数据读取 + Memory 多轮记忆 + Agent 自主工具决策 + Pydantic 结构化返回工业级智能体全链路，各组件协同完成 ReAct 思考执行闭环，完整模拟真实业务智能体开发流程。

2.2 基础智能体的三角架构

2.2.1 基础智能体三角闭环架构

所有最简无工具智能体统一由 Prompt 提示词、Model 模型、Result 结果三大模块构成循环闭环，三角流转逻辑如图 2.2-1 所示：

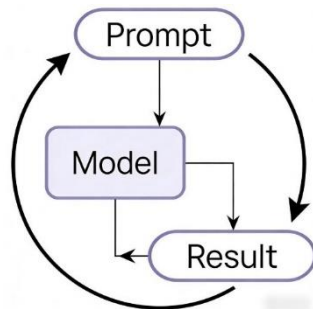


图 2.2-1：基础智能体三角闭环架构

三者完整流转逻辑：

(1) Prompt 提示词：承载全部角色定义、任务指令、输出约束，通过标准化消息封装后传递至模型，是控制模型行为的唯一人工入口；

(2) Model 模型：接收全部消息与提示内容完成推理、生成原始输出文本，是整个三角架构的运算核心；

(3) Result 结果：模型生成的原始应答，既作为对外展示内容，也可反向作为 AI 历史消息存入记忆，参与下一轮对话的提示词构建，形成循环优化闭环。

三者单向流转、循环迭代，任意智能体、固定链路都不会脱离该三角底层逻辑，所有工具、记忆、结构化输出均为三角架构的拓展衍生模块。

【例 2.2-1】三角架构极简农业问答智能体

【程序代码】

```
import os
# 1. 构建提示词
messages = [
    ("system", "你是专业农业技术专家，所有回答简洁精炼，不添加多余铺垫文字。"),
    ("human", "冬天下雪对来年农作物生长存在哪些有利影响？")
]

# 2. 初始化标准化模型
llm= ChatOpenAI(
    base_url=os.getenv("BASE_URL"),
    openai_api_key=os.getenv("OPENAI_API_KEY"),
    model=os.getenv("CHAT_MODEL"),
    temperature=0.1
)

# 3. 模型执行，完成三角架构流转
if __name__ == "__main__":
```

```
result = llm.invoke(messages)
print("三角架构标准化输出结果：")
print(result.content)
```

本例通过提示词—模型—结果构建了一个极简的智能体，通过结果可循环优化闭环。若需要优化输出效果，仅需要修改 `system` 系统消息内的提示词规则，无需调整模型与调用逻辑，直观体现 Prompt 提示词在三角架构中的核心调控作用。

2.2.2 消息与提示词

1、什么是提示词

提示词（Prompt）是连接开发者、用户与大模型的核心交互媒介，是 AI 可控开发的第一核心要素，没有标准化提示词，模型输出将完全随机、不可复用、无法落地工程场景。其核心作用可总结为四大核心价值：

- 角色定义：赋予模型专属身份与能力边界，明确模型可以做什么、禁止做什么，从根源约束模型行为，杜绝无效输出与幻觉内容。
- 任务指令：清晰传递用户核心需求、任务目标、执行步骤，解决模型语义理解偏差、需求拆解错误的问题。
- 格式约束：统一输出格式、排版规范、内容长度，让模型自由文本转化为标准化可复用内容，适配项目开发。
- 逻辑引导：引导模型遵循固定推理逻辑、思考步骤，提升答案精准度、逻辑性，适配复杂任务执行。

如【例 2.1-1】的提示词：

你是专业的 Python 编程助教。

【强制硬性规则，绝对不可违反】

1. 只允许回答 Python 编程、代码语法、Python 实操相关问题
2. 所有非 Python 相关问题，必须直接固定回复：抱歉，我仅可解答 Python 基础编程问题，不支持该类需求。
3. 禁止强行作答、禁止跨界创作、禁止自作主张回答无关内容

【例 2.1-2】的提示词：

你是专业天气预报助手，仅可调用 `get_weather_for_location` 工具查询城市天气，严格遵循规则：

1. 用户询问任意城市天气，必须调用工具获取实时数据，禁止编造气温、天气状况；
2. 工具返回结果后，按照指定结构化格式整理输出；
3. 无对应城市数据如实告知，禁止虚构天气信息；
4. 可结合上下文记忆回答用户追问。

可用工具：`get_weather_for_location`（输入中文城市名，返回近 3 天分段天气）

提示词是零成本优化模型效果的核心手段，无需微调模型、无需算力投入，仅通过话术优化即可大幅提升智能体精准度、可控性、稳定性。

2、什么是消息

（1）消息定义

消息是 LangChain 框架中和 LLM 交互的最小标准化单元，每条消息固定包含角色标识、文本内容、交互元数据三部分，区分对话来源，是提示词的标准化承载容器。普通纯文本

字符串无角色区分，无法留存多轮上下文；而结构化消息可精准划分系统、用户、AI、工具四类对话身份，支撑完整多轮交互记忆。

(2) 消息与提示词从属关系：

在 LangChain 及大语言模型（LLM）的交互体系中，提示词（Prompt）和消息（Message）是紧密关联但层次不同的概念，二者的关系可以概括为：消息是承载提示词的结构化载体，提示词是消息中传递给模型的核心指令内容。

- 提示词（Prompt）：传递给模型的指令、问题、上下文信息的总和。明确模型的任务目标，引导模型生成预期输出。如自然语言文本（写一篇关于 AI 的短文）。
- 消息（Message）：LangChain/LLM 交互中标准化的通信格式，包含角色、内容等元数据，区分对话角色（用户 / 工具 / 系统），携带上下文历史，让模型理解对话逻辑。如结构化对象（HumanMessage/AI Message）。

提示词是写入消息内部的指令文本，消息是承载提示词的结构化外壳；一段提示词不能脱离消息单独实现角色区分，一条消息内部必然包含对应提示内容。在 LangChain 的对话式交互中，用户输入的提示词会被封装成 HumanMessage，而模型的输出会被封装成 AI Message。

3、四大标准消息类型

LangChain 定义了四类标准交互消息，各类消息的优先级、生命周期等核心属性如表 2.2-1 所示：

表 2.2-1 四类消息核心区分对照表

消息类型	类名	优先级	生命周期	核心使用场景
系统消息	SystemMessage	最高	整段对话全程永久生效，不会随单次提问消失	定义智能体身份、硬性禁止行为、统一输出格式
用户消息	HumanMessage	中等	单次提问临时生效，本轮对话结束后转为历史记录	承载用户实时提问、补充需求、额外指令
AI 消息	AI Message	普通	存入记忆长期留存，仅作为上下文参考	存储模型上一轮完整应答内容
工具消息	ToolMessage	普通	单次工具调用后临时生效，本轮交互结束归档	存储网络、接口工具返回的原始观测数据

SystemMessage 优先级永久高于 HumanMessage，即使用户输入反向要求，模型也会优先执行系统内全部约束，这也是【例 2.1-1】可以拦截作文提问的底层原理。

4、消息三种标准化创建写法

(1) 文本提示

纯文本简易提示，仅适用于单轮无上下文极简问答，无法区分角色：

```
result = llm.invoke("写一段 Java 标准 HelloWorld 代码")
```

(2) 消息对象创建

创建结构化消息对象，规范工程实训标准写法，可读性强：

```
from langchain_core.messages import SystemMessage, HumanMessage
```

```

system_msg = SystemMessage("你是农业病虫害科普专家，回答简短通俗")
human_msg = HumanMessage("桔子黑点病如何提前预防？")
msg_list = [system_msg, human_msg]
result = llm.invoke(msg_list)

```

(3) 角色方式创建

你还可以用字典的格式来区分角色，消息通过 `role` 字段（如 `system/human/ai`）明确文本的来源，模型会根据角色差异处理提示词：

```

messages = [
    {"role": "system", "content": "你是农业病虫害科普专家，回答简短通俗"},
    {"role": "human", "content": "桔子黑点病如何提前预防？"}
]
result = llm.invoke(messages)

```

或者用元组格式来区分角色：

```

messages = [
    ("system", "你是农业病虫害科普专家，回答简短通俗"),
    ("human", "桔子黑点病如何提前预防？")
]
result = llm.invoke(messages)

```

5、AIMessage

模型调用后返回的 `AIMessage` 对象包含全套交互信息，除核心 `content` 应答文本外，还包含 `token` 消耗、调用 ID、工具调用标识等元数据，完整字段含义如表 2.2-2 所示：

表 2.2-2 AIMessage 元数据说明

属性名	类型	说明
<code>content</code>	<code>str / list</code>	AI 生成的核心内容： - 文本响应：直接为字符串 - 多模态响应：列表形式（如 <code>[{"type": "text", "text": "这是图片描述"}, {"type": "image_url", "image_url": "xxx"}]</code> ）。
<code>additional_kwargs</code>	<code>dict</code>	扩展参数字典，用于存储模型特定的额外信息： - 模型是否拒绝回答： <code>{'refusal': None}</code> - 工具调用： <code>{"function_call": {"name": "search", "arguments": {"query": "天气"}}}</code> ； - 多模态元数据： <code>{"image_width": 800, "image_height": 600}</code> ； - 模型自定义参数： <code>{"temperature": 0.7, "top_p": 0.9}</code> 。
<code>response_metadata</code>	<code>dict</code>	模型响应的元数据（LangChain 自动填充）： - <code>token_usage</code> : <code>token</code> 消耗（大模型按 <code>token</code> 计费的依据） - <code>completion_tokens</code> 模型生成回答消耗的 <code>token</code> 数（输出 <code>token</code> ） - <code>prompt_tokens</code> 你的提问消耗的 <code>token</code> 数（输入 <code>token</code> ）

		- total_tokens 总消耗 token 数（输入 + 输出） - id: 本次调用的唯一标识（可用于排查调用问题） - model_name: 生成模型名称（如 "Qwen/Qwen3-8B"） - response_time: 响应耗时（秒） - finish_reason: 模型停止原因
id	str	消息唯一标识（默认自动生成 UUID），用于对话历史追踪和去重。
tool_calls	list	模型生成的有效工具调用指令
invalid_tool_calls	list	模型生成的无效工具调用指令
usage_metadata	dict	LangChain 标准化的 token 消耗统计，和 response_metadata 中的 token_usage 内容一致

AIMessage 元数据中的 Token 是 AI 认识世界的最小“文字碎片”。就像我们人类学语言要先认识字一样，AI 大模型看不懂完整的句子，它只能看懂被切碎后的“最小意义单元”。这个最小单元，就是 Token。LLM 的训练、推理、上下文限制、成本计费，全部围绕 Token 展开。

举个具体的例子：

英文单词 “unhappiness” 可能被拆成 ["un", "happi", "ness"] 三个 Token

中文句子 “今天天气真好” 可能被拆成 ["今", "天", "天", "气", "真", "好"] 六个 Token

换算公式：一般来说，1 个 Token $\approx$ 1.5~2 个汉字。也就是说，一篇 1000 字的文章，大约会消耗 1500~2000 个 Token。

因为 AI 模型会区分消息的来源（是人发的、系统指令、还是 AI 自己发的），所以我们可以手动造一条 “AI 风格的回复” 插入对话记录，让模型后续处理时把这条 “假” 回复当成真实上下文，从而达到特定目的。比如你希望 AI 按固定话术回复，先手动插入一条符合要求的 AIMessage 到历史里，模型后续会参考这条 “假回复” 的风格 / 内容，输出更符合预期的结果。例如：

```
# 第二轮对话
if __name__ == "__main__":
    messages_again = [
        ("ai", "如果你要购买桔子，我会告诉你挑选没有黑点病桔子的方法"),
        ("human", "我想买 1 公斤桔子")
    ]
    result_again = llm.invoke(messages_again)
    print(result_again.content)
```

在提示词前添加一条 AIMessage，当模型处理 “我想买 1 公斤桔子” 这个问题时，会以为 “如果你要购买桔子，我会告诉你挑选没有黑点病桔子的方法” 这句话是它自己之前说的，它就会将挑选的方法告诉你。

如果在提示词前没有添加 AIMessage，它回复的问题就不会是挑选桔子的问题了，可能购买渠道和价格之类的信息。

6、消息模板

LangChain 的消息模板（Message Templates）是标准化 Prompt 结构、动态填充变量、

支持多角色对话（System/Human/Assistant）的核心工具，适用于单轮生成、多轮对话、工具调用等场景。

LangChain 中消息模板主要是 ChatPromptTemplate（对话型模板），支持动态变量填充、局部参数预填充，适配多场景批量生成需求，是工业级提示词标准化方案。

1) 消息点位符

使用 {变量名} 作为动态填充标记，运行时传入参数替换对应文本。

2) 创建模板方法

(1) from_template(): 只用单一段文本创建简易模板，适合单条简单提示。

```
prompt = ChatPromptTemplate.from_template("给我写{num}句{style}宣传语")
```

(2) from_messages(): 按 system、user 等多角色消息结构创建模板，适合多轮对话提示。

```
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是{type}专家，回答简短"),
    ("human", "讲解{topic}基础知识")
])
```

3) 填充变量方法

(1) format_messages(**kwargs): 一次性传入所有变量，完整渲染生成消息列表。

【例 2.2-2】农产品文案提示词模板

【程序代码】

```
from langchain_core.prompts import ChatPromptTemplate

# 分层模板，系统模板+用户模板，内置动态变量
system_template = "你是农产品电商文案师，生成{times}条{style}风格宣传文案，文字简短适配短视频平台"
human_template = "产品名称：{product_name}，核心卖点：{key_features}，目标受众：{target_audience}"

# 组装对话提示模板
chat_prompt = ChatPromptTemplate.from_messages([
    ("system", system_template),
    ("user", human_template)
])

if __name__ == "__main__":
    # 模式 1：完整填充全部变量
    full_msg = chat_prompt.format_messages(
        times=5,
        style="治愈温柔",
        product_name="赣南脐橙",
        key_features="果肉脆嫩汁水充足",
        target_audience="学生、中老年群体"
```

```
)
res1 = llm.invoke(full_msg)
print("脐橙文案：\n", res1.content)
```

(2) `format(**kwargs)`: 一次性传入所有变量，完整渲染生成消息字符串。

与 `format_messages` 区别如表 2.2-3 所示:

表 2. 2-3 `format_message` 与 `format` 的区别

特性	<code>.format()</code>	<code>.format_messages()</code>
返回值类型	字符串 (str)	消息对象列表 (List[BaseMessage])
返回内容	将模板中的所有变量替换后，直接拼接成一个完整的文本字符串。	将模板中的每一条消息（如 <code>system</code> 、 <code>human</code> ）分别格式化，并保留为独立的 <code>SystemMessage</code> 、 <code>HumanMessage</code> 等对象，最后以列表形式返回。
适用场景	当你的提示词是纯文本，或者你只需要一个完整的字符串用于简单任务（如直接传入 <code>create_agent</code> 的 <code>system_prompt</code> ）时。	当你的提示词包含多条不同角色的消息，并且你需要将这些结构化的消息传递给支持多轮对话的模型（如通过 <code>model.invoke(messages)</code> ）时。
支持的模板类型	主要适用于 <code>PromptTemplate</code> 和包含变量但没有“消息角色”概念的简单模板。	专为包含 (" <code>system</code> ", "...")、(" <code>human</code> ", "...") 等消息角色的 <code>ChatPromptTemplate</code> 设计。

`format_messages` 方式可以保留消息的角色（系统、用户、助手），让模型更清楚地理解上下文:

```
if __name__ == "__main__":
    full_msg = chat_prompt.format_messages(
        times=5,
        style="治愈温柔",
        product_name="赣南脐橙",
        key_features="果肉脆嫩汁水充足",
        target_audience="学生、中老年群体"
    )
    print("format_messages 填充: ")
    print(full_msg)
    msg = chat_prompt.format(
        times=5,
        style="治愈温柔",
        product_name="赣南脐橙",
        key_features="果肉脆嫩汁水充足",
        target_audience="学生、中老年群体"
    )
    print("format 填充: ")
    print(msg)
```

(3) `partial(**kwargs)`: 先固定一部分不变的变量，剩下的后续再补充。需要配合

`format_messages()`方法填充剩余部分的变量。

```
if __name__ == "__main__":
    # 模式 2: partial 局部预填充固定参数, 仅替换产品信息
    partial_prompt = chat_prompt.partial(times=5, style="热血励志")
    partial_msg = partial_prompt.format_messages(
        product_name="南丰蜜桔",
        key_features="清甜护肠胃",
        target_audience="上班族、宝妈"
    )
    res2 = llm.invoke(partial_msg)
    print("蜜桔文案: \n", res2.content)
```

(4) `invoke(dict)`: 传入字典批量渲染模板, 返回可直接给模型调用的消息结构。

```
if __name__ == "__main__":
    # 模式 3: invoke 字典批量填充变量
    dict_msg = chat_prompt.invoke({
        "times":5,
        "style":"简约高级",
        "product_name":"砂糖桔",
        "key_features":"回甘浓郁无渣",
        "target_audience":"日常自律人群"
    })
    res3 = llm.invoke(dict_msg)
    print("砂糖桔文案: \n", res3.content)
```

在大语言模型（尤其是 `LangChain` 等框架的对话式交互场景中），聊天模型的核心输入输出逻辑围绕结构化的消息对象展开：它接收由多条不同角色的消息组成的列表作为输入，并经过内部的语义理解、上下文推理和文本生成后，最终返回一条标准的 `AIMessage` 对象作为输出。

具体来说，输入的消息列表并非简单的文本拼接，而是包含了明确角色标识的结构化数据，常见角色包括 `SystemMessage`（系统指令，用于定义模型的行为准则和背景设定）、`HumanMessage`（用户输入，承载具体的问题或需求）、`AIMessage`（历史回复，保存模型之前的输出，支撑多轮对话上下文）。这些消息按照交互时序排列，共同构成模型理解任务的完整上下文。

而模型输出的 `AIMessage` 同样是结构化对象，不仅包含核心的文本回复内容，还可携带额外元数据（如生成时间、令牌消耗、工具调用意向等），既可以直接作为最终结果返回给用户，也能作为历史消息再次传入模型，实现多轮对话的连贯流转。

2.2.3 大语言模型

1、什么是大语言模型

大语言模型（LLM）是智能体的推理引擎，它们驱动智能体的决策过程，确定调用哪些工具、如何解释结果以及何时提供最终答案。大语言模型的功能除了文本生成，许多模型还支持：

- 工具调用：调用外部工具（如数据库查询或 API 调用）并在其响应中使用结果。

- 结构化输出：模型响应被限制为遵循定义的格式。
- 多模态：处理和返回除文本之外的数据，例如图像、音频和视频。
- 推理：模型执行多步推理以得出结论。

2、模型两种标准化初始化方案

LangChain 提供两套官方模型初始化接口，适配不同实训场景。

1) ChatOpenAI 专属初始化

专为 OpenAI 兼容第三方平台设计（硅基流动、阿里百炼等），参数完整、可控性强。

(1) 方法原型（类定义）

```
class ChatOpenAI(BaseChatModel):
    def __init__(
        self,
        model: str = "gpt-3.5-turbo",
        temperature: float = 0.7,
        max_tokens: Optional[int] = None,
        api_key: Optional[str] = None,
        base_url: Optional[str] = None,
        organization: Optional[str] = None,
        timeout: Optional[Union[float, Tuple[float, float]]] = None,
        max_retries: int = 2,
        streaming: bool = False,
        n: int = 1,
        frequency_penalty: float = 0.0,
        presence_penalty: float = 0.0,
        top_p: float = 1.0,
        response_format: Optional[Dict[str, str]] = None,
        seed: Optional[int] = None,
        **kwargs: Any,
    ) -> None:
        super().__init__(**kwargs)
```

(2) 参数说明

ChatOpenAI 核心参数如表 2.2-3 所示：

表 2.2-3ChatOpenAI 核心参数说明

参数名	类型	默认值	核心说明
model	str	"gpt-3.5-turbo"	指定使用的 OpenAI 模型，如 "gpt-4o"、"gpt-3.5-turbo-0125"
temperature	float	0.7	随机性控制，0 = 确定性输出，1 = 最大随机性： - 取值越高：生成越随机、发散（适合创意生成，如文案、剧本）； - 取值越低：生成越确定、精准（适合事实问答、代码生成）。
api_key	str	None	OpenAI API 密钥
base_url	str	None	自定义 API 端点（如对接 Azure OpenAI 或第三方代理）
max_tokens	int	None	生成回复的最大令牌数（未传时使用模型默认上限）
streaming	bool	False	是否开启流式输出（需配合 stream() 方法使用）
max_retries	int	2	API 调用失败时的重试次数
timeout	float/Tuple	None	请求超时时间（单值 = 总超时，元组 = (连接超时，读取超时)）
response_format	Dict	None	指定输出格式，如 {"type": "json_object"} 强制 JSON 输出

2) init_chat_model 通用初始化

init_chat_model 是 LangChain v1.0 新增的通用初始化函数，设计目标是统一各类聊天模型的初始化入口，即统一多厂商模型入口，支持文心、DeepSeek、OpenAI 等多类模型一键切换。

(1) 方法原型

```
def init_chat_model(  
    model: str | None = None,  
    *,  
    model_provider: str | None = None,  
    configurable_fields: Literal["any"] | list[str] | tuple[str, ...] | None = None,  
    config_prefix: str | None = None,  
    **kwargs: Any,  
    ) -> BaseChatModel
```

(2) 参数说明

init_chat_model 核心参数说明如表 2.2-4 所示：

表 2.2-4 init_chat_model 核心参数说明

参数名	类型	核心说明
-----	----	------

model	str	核心标识: - 字符串: 如 "gpt-3.5-turbo" (自动映射到 ChatOpenAI)、"ernie-4.0" (映射到文心) - 已实例化对象: 直接返回该对象
model_provider	str	模型提供商, 如果未在模型参数中指定。 支持的 model_provider 值及其对应的集成包如下: - openai -> langchain-openai - anthropic -> langchain-anthropic - deepseek -> langchain-deepseek - 如果未指定, 将尝试从模型名称推断 model_provider。 将根据以下模型前缀推断提供商: - gpt-... o1... o3... -> openai - deepseek... -> deepseek -
configurable_fields	Literal list tuple None	哪些模型参数可在运行时配置: - None: 无可配置字段 (即固定模型)。 - 'any': 所有字段均可配置。 - list[str] Tuple[str, ...]: 指定的字段可配置。 设置 configurable_fields="any" 意味着像 api_key、base_url 等字段可以在运行时更改, 这可能会将模型请求重定向到不同的服务/用户。
**kwargs	Any	传递给底层聊天模型 __init__ 方法的额外特定于模型的关键字参数。常见参数包括: - temperature: 用于控制随机性的模型温度。 - max_tokens: 输出令牌的最大数量。 - timeout: 等待响应的最长时间 (秒)。 - max_retries: 失败请求的最大重试次数。 - base_url: 自定义 API 端点 URL。

使用 `init_chat_model` 不需要指定供应商的 `base_url`, 根据模型前缀推断提供商。LangChain 支持所有主流模型提供商, 包括 OpenAI、Anthropic、Google、Azure、AWS Bedrock、DeepSeek 等。如 `model="deepseek:deepseek-chat"`, `init_chat_model` 会推断供应商为 `deepseek`:

```
llm = init_chat_model(
    openai_api_key=os.getenv("DEEPSEEK_API_KEY"),
    model="deepseek:deepseek-chat",
    temperature=0.9,
    max_tokens=1024
)
```

但是, 并非所有模型的提供商都能支持推断, 比如阿里百炼和硅基流动。LangChain 为这些模型提供商提供 OpenAI 兼容的 API, 需要手动为模型指定 `model_provider="openai"`。

```
model = init_chat_model(
```

```
base_url="https://api.siliconflow.cn",
openai_api_key=os.getenv("OPENAI_API_KEY"),
model="Qwen/Qwen3-8B",
model_provider="openai", # 关键新增参数：指定模型提供商为 openai
temperature=0.9,
max_tokens=1024
)
```

`init_chat_model` 可以通过指定 `configurable_fields` 来创建运行时可配置模型，这样可以在运行时对不同的提示词使用不同的模型配置：

```
llm = init_chat_model(
    base_url="https://api.siliconflow.cn",
    openai_api_key=os.getenv("OPENAI_API_KEY"),
    model_provider="openai",
    temperature=0.9,
    max_tokens=1024
)
.....
configurable = {"model": "Qwen/Qwen3-8B"}
result = llm.invoke(filled_prompt, config = {"configurable": configurable})
```

ChatOpenAI 与 `init_chat_model` 选择：

- ChatOpenAI 是 LangChain 中对接 OpenAI 聊天模型的专属核心类，参数精准对应 OpenAI API，适合明确使用 OpenAI 模型的场景。
- `init_chat_model` 是 LangChain 提供的通用初始化函数，支持通过字符串快速创建任意厂商的聊天模型实例，适合多模型适配、快速开发的场景。

LangChain 智能体三角架构中的 LLM 作为整个交互链路的核心推理大脑，是可适配提示词且支持上下文感知的智能交互核心，它能够解析结构化消息、识别系统约束与用户需求，自主判断是否需要调用外部工具，按不同角色、业务场景输出规范适配性回复，统筹完成任务拆解、逻辑推理、结果整合全流程，大幅提升复杂任务的处理效率与人机交互体验。

2.3 工程级典型智能体

2.3.1 典型智能体完整运行流程

1、典型智能体的整体运行流程

典型智能体将大语言模型与工具结合，创建能够对任务进行推理、决定使用哪些工具并迭代地解决问题的系统。图 2.3-1 是一个典型的智能体的整体运行流程示意图，展示了一个基于“感知 - 决策 - 执行”逻辑的系统运行过程：

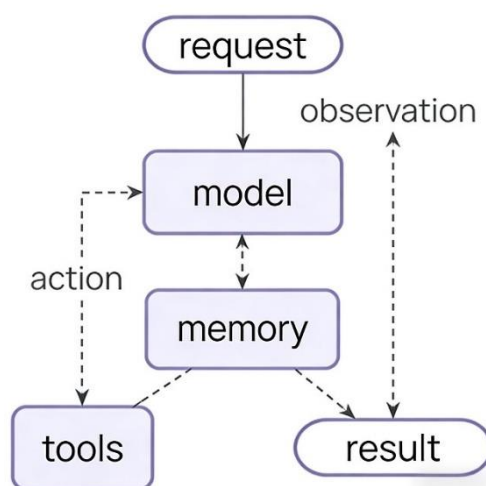


图 2.3-1：典型的智能体的整体运行流程

运行流程拆解说明：

（1）request（用户请求）

流程起始输入项，由用户输入自然语言问题、业务任务构成，会统一封装为标准化 HumanMessage 消息，作为整个智能体循环的初始输入源。

（2）model（大模型推理核心）

智能体的决策大脑，同时接收当前用户请求与 memory 存储的全部历史对话消息；结合系统提示完成语义理解、任务判断、路径规划，自主生成下一步行动指令：若无需外部数据则直接生成最终应答，若需要实时外部信息则输出工具调用指令 action。

（3）memory（对话记忆存储模块）

智能体上下文存储载体，双向对接模型：一方面接收 model 产生的所有消息（用户提问、AI 回复、工具返回内容）并持久保存；另一方面在每一轮推理前，向 model 推送完整历史交互数据，为模型决策提供上下文支撑，实现连贯多轮对话。

（4）tools（外部工具执行集）

承接模型下发的 action 行动指令，是智能体获取外部实时数据、执行实操动作的执行单元。当模型判断需要外部信息或执行特定动作时，就会调用工具（比如数据库查询、计算器、API 调用等）。工具执行完毕后，将原始返回数据封装为 observation 观测信息回传给模型，补充推理所需外部真实信息。

（5）result（标准化输出结果）

当 model 整合用户提问、历史记忆、工具观测数据，判定信息充足时生成最终结构化应答，对外反馈给用户；同时本次完整交互的全部消息统一存入 memory 留存，观测信息同步回传模型，完成一轮完整执行闭环。

2、创建智能体

在 LangChain 中，`create_agent` 是构建智能体（Agent）的核心方法，智能体能够根据用户输入 自主决策调用工具，完成复杂任务。

1) 方法原型

```
create_agent(  
    model: str | BaseChatModel,  
    tools: Sequence[BaseTool | Callable | dict[str, Any]] | None = None,  
    *,  
    system_prompt: str | None = None,  
    middleware: Sequence[AgentMiddleware[AgentState[ResponseT], ContextT]] = (),  
    response_format: ResponseFormat[ResponseT] | type[ResponseT] | None = None,  
    state_schema: type[AgentState[ResponseT]] | None = None,  
    context_schema: type[ContextT] | None = None,  
    checkpointer: Checkpointer | None = None,  
    store: BaseStore | None = None,  
    interrupt_before: list[str] | None = None,  
    interrupt_after: list[str] | None = None,  
    debug: bool = False,  
    name: str | None = None,  
    cache: BaseCache | None = None,  
) -> CompiledStateGraph[  
    AgentState[ResponseT], ContextT, _InputAgentState, _OutputAgentState[ResponseT]  
]
```

2) 核心参数说明

`create_agent` 的核心参数如表 2.3-1 所示：

表 2.3-1 `create_agent` 核心参数说明

参数名	类型	核心作用
model	LLM/ChatModel 实例	用于智能体的语言模型。可以是一个字符串标识符（例如，"openai:gpt-4"）或一个直接的聊天模型实例（例如，ChatOpenAI 或其他聊天模型）。
tools	工具列表	智能体可调用的工具。
system_prompt	字符串 / ChatPromptTemplate	一个可选的用于 LLM 的系统提示。提示会被转换为一个 SystemMessage 并添加到消息列表的开头。
context_schema	Pydantic BaseModel	结构化输入 schema，约束用户输入的格式。
response_format	Pydantic BaseModel	结构化输出 schema，强制智能体按固定格式返回结果。
checkpointer	Checkpointer	一个可选的检查点保存器对象。用于为单个线程（例如，单个对话）持久化图的状态（例如，作为聊天记忆）。

store	存储实例（如 VectorStore/Redis）	一个可选的存储对象。 用于在多个线程（例如，多个对话/用户）之间持久化数据。
-------	---------------------------	---

【例 2.3-1】一个能调用外部工具的典型的智能体。

本案例基于 `create_agent` 搭建双工具多功能智能体，解决原生 LLM 两大核心缺陷：一是模型训练数据存在时间截止点，无法获取实时日期；二是模型纯文本估算字数容易产生幻觉误差。程序内置两类自定义工具，搭配高优先级系统提示约束模型行为，同时设置两组对照测试，直观对比原生 LLM 与具备自主工具调度能力的智能体的执行差异。

【程序代码】

```
import datetime
from langchain.agents import create_agent
from langchain.tools import tool

# 工具 1：实时日期获取工具（解决模型知识滞后）
@tool
def get_current_datetime() -> str:
    """
    获取当前系统标准北京时间，包含年月日、星期
    用于回答所有时间、日期、年份相关提问
    返回：标准化格式化时间字符串
    """
    week_list = ["星期一", "星期二", "星期三", "星期四", "星期五", "星期六", "星期日"]
    now = datetime.datetime.now()
    week = week_list[now.weekday()]
    return f"当前时间：{now.strftime('%Y 年 %m 月 %d 日')} {week}"

# 工具 2：文本字数统计工具（自定义业务工具）
@tool
def text_word_count(text: str) -> str:
    """
    统计输入文本的有效字数（不含空格、换行）
    参数：text - 需要统计的原始文本字符串
    返回：文本有效字数统计结果
    """
    if not text:
        return "输入文本为空，无法统计字数"
    clean_text = text.replace(" ", "").replace("\n", "")
    return f"文本有效字数为：{len(clean_text)}字"

# 系统规则约束（最高优先级）
SYSTEM_PROMPT = """
你是多功能实用助手，严格遵循以下规则：
1. 时间、日期提问必须调用 get_current_datetime 工具，禁止编造时间
```


2. 字数统计需求必须调用 `text_word_count` 工具，精准计算
3. 非工具类问题直接自主回答，简洁精准、无冗余内容
4. 杜绝 AI 幻觉，所有工具数据如实反馈

"""

组装多工具智能体

```
agent = create_agent(  
    model=llm,  
    tools=[get_current_datetime, text_word_count],  
    system_prompt=SYSTEM_PROMPT  
)
```

综合场景测试

if __name__ == "__main__":

测试一：不用智能体

result = llm.invoke("帮我查一下今天日期，再统计这句话的有效字数：人工智能让开发更高效")

print("LLM 单场景执行结果：\n", result)

测试二：多场景混合提问，验证自主工具决策能力

```
result = agent.invoke({  
    "messages": [  
        "帮我查一下今天日期，再统计这句话的有效字数：人工智能让开发更高效"  
    ]  
})
```

print("智能体综合执行结果：\n", result["messages"][-1].content)

【程序模块拆解说明】

①自定义工具模块

采用 `@tool` 装饰器实现两个可被模型自动识别的工具：

`get_current_datetime`：读取系统本地实时时间，格式化输出年月日 + 星期，专门处理日期、时间类提问，杜绝模型编造时间；

`text_word_count`：清洗文本空格、换行后精准统计有效字符，解决模型估算字数不准确的幻觉问题；

②系统提示词模块

作为全局最高优先级 `SystemMessage`，明确四条硬性执行规则：区分两类问题对应的专属工具调用要求、普通问题直接作答、禁止编造虚假数据，全程约束模型推理逻辑，从规则层面规避 AI 幻觉。

③智能体组装模块

通过 `create_agent` 整合模型、双工具集合、系统提示，赋予模型自主判断需求、动态选择对应工具的能力，形成“推理—调用工具—获取真实数据—整合答案”的完整闭环。

④两组对照测试逻辑

测试一：直接调用原生 LLM，无工具调度能力，作为对照组；

测试二：调用完整智能体，一条提问同时包含日期查询、字数统计两类工具需求，验

证智能体可自主识别多任务、依次调用对应工具、整合两份工具结果统一输出。

【程序运行结果解读】

①原生独立 LLM 仅具备文本生成能力，无法对接外部实时资源、完成精确数值计算，存在严重幻觉问题；

②搭配自定义 Tools + 智能体架构后，模型获得自主调度外部能力的权限，可按需调用工具获取客观真实数据；

③系统提示词是约束工具调用逻辑的关键，能强制模型在对应场景下必须使用工具，保障输出可信度；

④智能体支持复合多任务拆解，可自动识别一条提问内多个工具需求，分步执行并整合结果，交互更贴合人类自然提问习惯。

3、智能体执行结果与查看方式

1) 智能体执行消息存储规则

调用 `agent.invoke()` 运行智能体后，返回结果中的 `messages` 列表会严格按照交互时间顺序，完整存储本次调用全程产生的所有消息，完整记录“用户输入—AI 工具调用行为—工具返回数据—AI 最终答复”的全流程，列表内包含三类核心信息：

- 用户原始提问内容；
- AI 的推理记录与工具调用指令日志；
- 整合全部信息后生成的最终应答文本。

2) 打印全流程消息

以【例 2.3-1】智能体为例，通过循环调用消息内置格式化方法，完整打印每一轮交互记录：

```
if __name__ == "__main__":  
  
    result = agent.invoke({  
        "messages": [  
            "帮我查一下今天日期，再统计这句话的有效字数：人工智能让开发更高效"  
        ]  
    })  
    for m in result["messages"]:  
        m.pretty_print()
```

标准完整输出展示：

```
===== Human Message =====  
帮我查一下今天日期，再统计这句话的有效字数：人工智能让开发更高效  
===== Ai Message =====  
Tool Calls:  
  get_current_datetime (019f89e09eeefa0b67961f0fbd5fdad)  
Call ID: 019f89e09eeefa0b67961f0fbd5fdad  
Args:  
  text_word_count (019f89e09eeefa0b67961f0fbd5fdae)  
Call ID: 019f89e09eeefa0b67961f0fbd5fdae  
Args:  
  text: 人工智能让开发更高效
```

===== Tool Message =====

Name: get_current_datetime

当前时间: 2026 年 07 月 22 日 星期三

===== Tool Message =====

Name: text_word_count

文本有效字数为: 10 字

===== Ai Message =====

今天是 2026 年 07 月 22 日 星期三, "人工智能让开发更高效"这句话的有效字数为 10 字。

【执行结果解读】

invoke()方法的返回值并非仅输出本次对话的最终一句话,而是存储了从本次提问发起至应答完成的完整消息数组。完整链路依次为:用户 Human 提问消息—携带工具调用标识的 AI 消息—两条工具返回 Tool 消息—整合信息后的最终 AI 应答消息。

将全部消息统一存入列表,能够完整追溯智能体全部思考、工具调用、数据返回环节,方便调试排查工具不执行、参数传错、输出异常等问题。

3) 三类常用消息读取方式

(1) 打印全部完整交互消息

所有消息类型 (HumanMessage/AI Message/ToolMessage/SystemMessage) 都内置 pretty_print() 格式化打印方法,相比原生 print(),可以自动区分消息类型、清晰展示调用 ID、入参、返回内容,是 LangChain 专用调试方式:

```
for m in result["messages"]:  
    m.pretty_print()
```

(2) 单独提取用户原始提问

消息列表首位固定为用户输入的 HumanMessage,通过下标 0 读取提问文本:

```
result["messages"][0].content
```

(3) 单独提取最终回复内容

经过工具调用、数据整合后的最终 AI 应答永远存放在 messages 列表末尾,使用下标-1 快速获取结果:

```
result["messages"][-1].content
```

2.3.2 Tools 工具

大语言模型的知识全部来源于训练阶段的数据集,模型固化的训练数据存在时间截止点,无法获取实时、动态外部信息,也不能主动执行各类程序操作。想要突破模型自身能力局限,拓展模型可处理任务的边界,LangChain 提供的 Tools 工具机制是标准解决方案。

工具机制的核心逻辑是由大模型自主判断需求,主动调用外部 API、本地函数、第三方服务,获取真实外部数据并交给模型整合分析,以此弥补模型静态知识库的短板。

1、工具的作用

LangChain 工具作为智能体对接外部世界的标准载体,核心能力分为三类:

(1) 连接各类外部资源:可访问数据库、实时公开 API (天气、日期、股票等)、本地文本文件、网页文本;

(2) 执行实体操作任务:运行 Python 计算代码、发送 HTTP 网络请求、调用计算

器、读写 Excel 表格；

(3) 消除 AI 幻觉，提升决策准确度：模型不再单纯依靠训练数据凭空编造内容，可依托工具返回的客观真实数据生成回答，大幅降低虚假输出概率。

【例 2.3-2】无工具智能体：直观展示模型“幻觉”问题

【程序代码】

```
if __name__ == "__main__":
    result = llm.invoke("今天是几号？")
    print(result.content)
```

【执行结果解读】

原生大模型无法获取设备实时系统时间，训练数据集仅包含历史时间信息。即便模型能够输出一个日期数字，该结果大概率与真实当前日期不符，这种模型凭空编造不实信息的现象称为 AI 幻觉。

想要解决该问题，为模型补充获取实时信息的能力，就需要引入 Tools 工具机制，给模型提供可调用的外部数据获取渠道。

2、LangChain 工具标准使用闭环流程

LangChain 智能体调用工具遵循模型决策—工具调用—数据回传—结果整合标准化闭环，整套流程依赖四大核心组件协同工作：

- Tool 工具：封装单一完整功能的函数，如日期读取、天气查询、数值计算；
- Agent 智能体：整体调度核心，自主判断是否需要调用工具、选用哪一款工具、填充哪些入参；
- LLM 大模型：驱动智能体完成语义理解、需求拆解、工具参数生成的推理核心；
- Toolkit 工具集：将功能相近的多款工具打包封装，例如数据库工具集包含连接、查询、插入工具。

3、自定义工具创建规范

(1) @tool 装饰器

LangChain 创建自定义工具最简便、最通用的方式是使用 @tool 装饰器封装 Python 函数，装饰器从 langchain.tools 包导入 LangChain。

默认规则：工具名称自动复用函数名；工具描述直接读取函数内部文档注释（docstring），大模型依靠这段注释判断工具适用场景、入参要求，因此工具注释必须清晰完整。

写法 1：基础无参装饰器

```
@tool
def get_current_datetime() -> str:
    """
    获取当前系统标准北京时间，包含年月日、星期
    用于回答所有时间、日期、年份相关提问
    返回：标准化格式化时间字符串
    """
```

写法 2：自定义工具名称

如需修改模型识别的工具名称，可在装饰器中传入字符串参数重命名：

```
@tool("get_today_date")
```

写法 3：精细化参数描述工具

工具支持接收自定义入参，需在函数注释中清晰标注参数名称、参数类型、参数含义、示例，大模型可自动识别参数结构并完成参数填充，如【例 2.1-2】的天气工具：

```
@tool
def get_weather_for_location(city: str) -> str:
    """
    获取指定中文城市近 3 天天气预报
    参数 city: 国内/海外城市中文名称，如南昌、深圳、北京
    返回：标准化天气 JSON 文本，包含实时温度、昼夜气温、天气状况
    """
```

当用户提问南昌今天的天气如何时，智能体会自动提取“南昌”作为 city 参数传入工具执行。

(2) 工具开发注意事项

- 函数参数必须添加类型注解，框架可自动识别参数结构生成参数校验规则 reference....;
- 文档注释是模型识别工具的唯一依据，注释模糊会导致智能体不会调用、传参错误；
- 工具内部建议增加异常捕获逻辑，接口超时、参数错误时返回友好提示文本，避免程序中断。

【例 2.3-3】搭建气象咨询专用智能体

本案例基于 LangChain 框架搭建气象咨询专用智能体，全部网络接口统一使用国内平台接口盒子（apihz.cn）免费 API，实现三大核心能力：自动获取本地系统日期、通过公网 IP 自动定位用户所在城市、输入城市名称查询当日天气预报。

① 依赖包导入

```
import datetime
import requests
from langchain.tools import tool
from langchain.agents import create_agent
```

② 三大工具模块

工具 1：get_current_date 获取系统当前日期

功能：读取本地系统时间，匹配中文星期，输出标准化日期字符串

触发规则：用户提问未指定查询日期时，智能体自动调用

```
@tool
def get_current_date() -> str:
    """
    获取本机系统当前年月日、星期；用户提问未指定查询日期时自动调用
    返回：格式化日期字符串
    """

    week_list = ["星期一", "星期二", "星期三", "星期四", "星期五", "星期六", "星期日"]
    now = datetime.datetime.now()
    week = week_list[now.weekday()]
    return f"当前日期：{now.strftime('%Y 年%m 月%d 日')} {week}"
```

工具 2: get_current_city 自动 IP 定位城市（双层嵌套结构）

该工具分为内层辅助函数 get_current_ip、外层智能体工具 get_current_city。

内层函数 get_current_ip（非 @tool 工具，仅内部调用）

```
def get_current_ip() -> str:
    """
    获取公网 IP
    返回：公网 IP；定位失败则返回引导文本
    """
    try:
        # 获取公网 IP，需要到接口盒子 https://apihz.cn/api/chaip.html 去申请 id 和 key
        url = "https://cn.apihz.cn/api/ip/getapi.php?id=10019433&key=fe034a161aeec2bf45f841b6024e898b"
        ip_res = requests.get(url, timeout=8)
        ip_res.raise_for_status()
        data = ip_res.json()
        if data.get("code") != 200:
            return "无法获取公网 IP"
        return data.get("ip")
    except Exception as err:
        return "无法获取公网 IP： {str(err)}"
```

外层工具 get_current_city（@tool 标记，智能体可直接调用）

执行逻辑：先调用 get_current_ip 拿到公网 IP，再将 IP 传入归属地查询 API

解析接口返回 shi 字段（城市名称），成功则返回城市名；接口异常、无城市数据则返回友好失败提示

触发规则：用户提问未写明查询城市时，智能体自动调用，实现无需用户手动输入城市即可查询本地天气

```
@tool
def get_current_city() -> str:
    """
    用户提问未提供城市时自动调用，通过公网 IP 获取归属城市
    返回：IP 归属城市；定位失败则返回引导文本
    """
    try:
        # 获取公网 IP
        ip = get_current_ip()
        # 获取归属城市
        url = f"https://cn.apihz.cn/api/ip/chaapi.php?id=10019433&key=fe034a161aeec2bf45f841b6024e898b&ip={ip}"
        ip_res = requests.get(url, timeout=8)
        ip_res.raise_for_status()
        data = ip_res.json()
        if data.get("code") != 200:
```

```

        return "未知城市"
    city = data.get("shi")
    if city:
        return city
    else:
        return "未知城市"
except Exception as err:
    return f"自动定位城市失败: {str(err)}"

```

工具 3: `get_weather_by_city_name` 城市天气查询工具

功能: 根据中文城市名称直接查询 7 日天气预报

```

@tool
def get_weather_by_city_name(city: str) -> str:
    """
    根据中文城市名称直接查询 7 日天气预报
    参数 city: 中文城市名称, 例如南昌、北京、上海
    返回: 当日天气
    """
    url = f"https://cn.api.hz.cn/api/tianqi/tqyb.php?id=10019433&key=fe034a161aee2bf45f841b6024e898b&place={city}&day=7"
    try:
        res = requests.get(url, timeout=10)
        res.raise_for_status()
        data = res.json()
        # 接口返回非 200 代表查询失败
        if data.get("code") != 200:
            return f"天气查询失败: {data.get('msg')}, 请确认城市名称正确"

        return data
    except Exception as err:
        return f"气象接口请求异常, 查询失败: {str(err)}"

```

③系统提示词 `SYSTEM_PROMPT` 规则

提示词是智能体最高优先级执行约束, 定义 5 条硬性运行规则, 管控智能体全部行为:

```
SYSTEM_PROMPT = """
```

你是专业气象咨询助手, 严格遵守以下执行规则:

1. 处理用户天气提问时, 自动判断缺失信息:

- 1.1 提问未写明查询城市: 自动调用 `get_current_city` 获取本地城市
- 1.2 提问未写明查询日期: 自动调用 `get_current_date` 获取当天日期

2. 天气查询固定流程:

仅需调用 `get_weather_by_city_name` 工具, 直接传入城市名称即可查询天气

3. 数据真实性约束:

禁止编造城市、日期、气温、天气状况, 所有气象数据必须通过工具获取

4. 异常处理规则:

工具返回报错、定位失败、城市不存在等信息，直接原样告知用户，不得自行编造修正

5. 输出规范：

最终回答整合三项信息：查询日期、目标城市、当日完整天气预报（天气、气温、风向、风级）

），回答清晰完整。

"""

④智能体组装

```
agent = create_agent(  
    model=llm,  
    tools=[get_current_date, get_current_city, get_weather_by_city_name],  
    system_prompt=SYSTEM_PROMPT  
)
```

⑤测试入口（main 函数）

程序提供三组典型用户提问测试用例，覆盖全部业务场景，验证智能体自动补全缺失信息、工具自主调度能力：

```
if __name__ == "__main__":  
    # 测试 1：指定城市，无日期，自动补全今日日期  
    msg1 = ["查询南昌的天气"]  
    res1 = agent.invoke({"messages": msg1})  
    print("====测试 1 输出====\n", res1["messages"][-1].content)  
    print("-" * 80)  
  
    # 测试 2：无城市、无日期，自动 IP 定位城市+自动获取今日日期  
    msg2 = ["天气怎么样"]  
    res2 = agent.invoke({"messages": msg2})  
    print("====测试 2 输出====\n", res2["messages"][-1].content)  
    print("-" * 80)  
  
    # 测试 3：同时指定城市与日期(7 天内)  
    msg3 = ["查询 2026 年 07 月 30 日北京的天气"]  
    res3 = agent.invoke({"messages": msg3})  
    print("====测试 3 输出====\n", res3["messages"][-1].content)
```

【程序执行结果解读】

本程序基于 LangChain @tool 装饰器封装三类独立工具，通过 create_agent 构建具备自主推理、工具自动调用能力的气象智能体，实现自动化交互设计：用户模糊提问无需补充参数，智能体自动获取本地城市、今日日期，降低用户操作成本。并依靠系统提示词约束 AI 行为，从根源杜绝 AI 幻觉、编造数据。

LangChain 智能工具是由 Agent 自主调度、封装单一业务能力、内置异常捕获的可调用功能组件。引入工具后，模型可获取真实客观的外部实时数据，从根源减少 AI 幻觉；同时大幅拓展模型可处理任务范围，显著提升复杂业务任务的处理准确度与稳定性。

2.3.3 Memory 记忆

记忆是 LangChain 智能体用于存储全部历史交互信息的存储系统。对 AI 智能体而言，记忆是实现连贯多轮对话的核心基础：智能体可读取过往问答内容理解上下文、根据用户历史交互适配回答风格、从多轮反馈中调整输出逻辑。当智能体处理多轮连续提问、复杂分步任务时，记忆能力直接决定交互流畅度与用户体验。

LangChain 记忆系统模拟人类记忆模式，分为短期会话记忆与长期持久化记忆两大类：

- 短期记忆（临时内存记忆）：仅保存在程序内存中，仅在单条会话线程内生效；程序关闭、服务重启后所有对话记录全部丢失，适合单次调试场景。
- 长期记忆（持久化存储记忆）：将对话、用户关键信息持久存入数据库，支持跨程序重启、跨会话检索历史交互内容，适用于线上生产部署场景。

1、短期内存记忆 InMemorySaver

短期记忆依托 InMemorySaver 实现，存储介质为程序运行内存。使用规则：

（1）创建智能体时必须传入 `checkpointer=InMemorySaver()` 开启记忆。

```
agent = create_agent(  
    checkpointer=InMemorySaver(),  
)
```

（2）LangChain 记忆基于线程隔离，调用 `agent.invoke()` 时必须配置唯一 `thread_id` 线程标识。同一 `thread_id` 下的所有对话消息会自动存入内存，实现上下文连贯读取，不同 `thread_id` 会话完全隔离，无法互相读取历史对话。

```
agent.invoke(  
    {"configurable": {"thread_id": 线程 ID}},  
)
```

【例 2.3-4】创建具有短期记忆能力的智能体

本程序基于 LangChain 的 InMemorySaver 内存型检查点，搭建拥有短期上下文记忆的农业专家智能体。依靠 `thread_id` 区分独立对话会话，同一标识下多轮对话会自动缓存历史聊天记录，实现上下文连贯问答。记忆仅存于程序运行内存，程序关闭则记忆全部清空。

【程序代码】

```
# 导入依赖  
from langgraph.checkpoint.memory import InMemorySaver  
  
SYSTEM_PROMPT = "你是资深农业技术专家,仅解答农业相关问题，回答简洁准确"  
# 初始化内存检查点，开启短期记忆  
agent = create_agent(llm, system_prompt=SYSTEM_PROMPT,  
    checkpointer=InMemorySaver())  
  
if __name__ == "__main__":  
    # 第一轮提问  
    messages_1 = ["列举 3 种水稻的易发病虫害"]  
    result_1 = agent.invoke(  
        {"messages": messages_1},  
        {"configurable": {"thread_id": "1"}}  
    )
```

```
print(result_1["messages"][-1].content)
print("="*50)
# 第二轮上下文追问，自动读取上一轮对话记忆
messages_2 = ["在这 3 种病虫害中，告诉我第二种病虫害的防治方法"]
result_2= agent.invoke(
    {"messages": messages_2},
    {"configurable": {"thread_id": "1"}}
)
print(result_2["messages"][-1].content)
```

【程序执行结果解读】

两轮提问使用同一个 thread_id=1，内存自动保存第一轮完整对话消息；第二轮用户仅模糊追问，智能体读取历史上下文，无需用户重复描述需求，完整实现多轮连贯对话。

2、长期持久化记忆 RedisSaver

短期内存记忆存在明显短板：程序重启、会话结束后所有对话记录直接丢失，无法留存长期用户交互数据。长期记忆通过数据库持久化存储对话检查点，并能通过检索召回。

1) 核心原理

长期记忆的核心原理是存储与检索：

- (1) 将用户的关键信息 / 历史对话嵌入为向量，存储到向量数据（可以基于 Redis、MongoDB、PostgreSQL 等数据库来保存聊天记录）；
- (2) 新对话时，检索向量库中与当前问题相关的长期记忆；
- (3) 将短期上下文 + 检索到的长期记忆一起传入大语言模型。

2) 长期记忆的实现方式

长期记忆存储方案推荐使用 Redis-stack 数据库实现持久化会话记忆。Redis 8.0 及以上版本支持向量结构，可存储对话向量、会话检查点数据，具备亚毫秒级读写延迟，支持实时对话 AI、推荐系统等高实时性业务场景。

(1) RedisSaver 类

LangChain 官方提供了 RedisSaver 类，是 Redis 持久化检查点工具。RedisSaver 类的核心参数说明如表 2.3-2 所示：

表 2.3-2 RedisSaver 核心参数说明

参数	类型	核心作用
redis_url	str	Redis 连接 URL（与 redis_client 二选一）
redis_client	Redis/RedisCluster	预先初始化完成的 Redis 客户端，推荐复用该方式
ttl	Dict[str, Any]	设置会话检查点过期时长，自动清理长期闲置会话
checkpoint_prefix	str	检查点键前缀，区分多项目会话数据，避免键冲突

(2) 使用规则

使用 RedisSaver 类创建检查点：

```
# 创建 RedisSaver 检查点
checkpointer = RedisSaver(
```

```
    redis_url= redis_url
)
```

RedisSaver 运行需要搜索索引 checkpoint_write 等，必须在创建 RedisSaver 实例后调用 setup() 方法，自动在 Redis Stack 中创建 RedisSaver 运行所需的所有搜索索引：

```
# 创建 checkpoint_write 等索引
checkpointer.setup()
```

【例 2.3-5】创建具有长期记忆能力的智能体

本案例基于 LangChain 构建具备 Redis 持久化记忆的农业专家智能体，核心依靠 RedisSaver 检查点实现会话上下文持久化存储，采用 redis_url 统一连接 Redis 服务，通过唯一 thread_id 隔离独立对话会话，完整实现多轮上下文连贯问答，同时设置 24 小时会话过期策略，适配生产环境持久化需求。

【程序代码】：

```
# 1. 导入 Redis 持久化检查点工具
from langgraph.checkpoint.redis import RedisSaver

# 2. Redis 连接 URL（密码=123456，host:192.168.100.101 port:6379 db0）
redis_url = "redis://:123456@192.168.100.101:6379/0"

# 3. 直接用 redis_url 参数实例化 RedisSaver，创建检查点
checkpointer = RedisSaver(
    redis_url=redis_url,
    checkpoint_prefix="agri_agent_checkpoint_",
    ttl={"checkpoint": 3600 * 24} # 会话数据保留 24 小时
)

# 自动创建 Redis 检索索引
checkpointer.setup()

# 4. 组装带 Redis 长期记忆的智能体
SYSTEM_PROMPT = "你是资深农业技术专家,仅解答农业相关问题，回答简洁准确"
agent = create_agent(
    model= llm,
    system_prompt=SYSTEM_PROMPT,
    checkpointer=checkpointer # 绑定 Redis 持久化存储
)

# 5. 多轮对话测试
if __name__ == "__main__":
    # 第一轮提问
    messages_1 = ["列举 3 种水稻的易发病虫害"]
    result_1 = agent.invoke(
        {"messages": messages_1},
        {"configurable": {"thread_id": "1"}})
```

```
)
print(result_1["messages"][-1].content)
print("="*50)
# 第二轮上下文追问
messages_2 = ["在这 3 种病虫害中，告诉我第二种病虫害的防治方法"]
result_2= agent.invoke(
    {"messages": messages_2},
    {"configurable": {"thread_id": "1"}}
)
print(result_2["messages"][-1].content)
```

【程序执行结果解读】

程序执行分为三层逻辑：Redis 存储初始化—绑定持久化检查点构建智能体—两轮连续对话测试，将对话数据存储于远端 Redis 服务，而非程序内存，程序关闭、重启后，再次使用 thread_id="1"调用智能体，可完整读取两轮历史对话，延续上下文。智能体每轮对话结束自动完成状态快照存入 Redis，完整记录用户提问、AI 回答、中间推理状态，支持后续回溯历史对话排查问题。

LangChain 智能体的短期记忆依托内存实时保留当前交互的对话信息，长期记忆通过向量数据库持久化存储跨会话的关键知识与历史交互记录。一个具有记忆能力的智能体既能保障当前对话的连贯性与上下文一致性，又能沉淀可复用的领域知识，大幅提升智能体在复杂任务中的决策准确性与多轮交互能力。

2.3.4 结构化输出

在 AI 应用开发场景中，大语言模型（LLM）原生输出内容为无规范约束的非结构化自然语言。若直接将这类文本用于程序逻辑处理，例如数据入库、接口参数传递、工具函数调用等场景，需要额外编写文本解析逻辑，不仅开发效率低下，还极易出现解析异常、数据错乱等问题。

LangChain 提供了一套成熟完整的结构化输出解决方案，可约束智能体按照开发者指定、格式可预测的标准返回数据。开发者无需手动编写自然语言解析代码，可直接获取 JSON 字符串、Pydantic 模型实例、数据类等标准化结构化数据，可直接在业务程序中读取、运算、存储。

1、Pydantic 模型基础

Pydantic 是 Python 生态中用于强类型数据校验、结构化数据定义的工具库，同时也是 LangChain 实现高精度结构化输出的底层核心依赖。

其核心作用是预先定义数据规范：包含字段名称、数据类型、数值、文本约束规则，同时自动对输入输出数据执行合法性校验，完美适配 LLM 标准化输出的业务需求。

对于编程初学者，可将 Pydantic 模型理解为三层能力载体：

（1）标准化数据说明书：强制规定数据必须包含的全部字段、每个字段对应数据类型（数字 / 字符串 / 列表等）、字段约束条件（例如价格数值必须大于 0）；

（2）全自动数据校验器：自动校验数据是否匹配预设规范，不合规数据直接抛出异常；同时支持自动类型转换，例如自动将字符串"9999"转换为数字 9999；

（3）便捷结构化数据容器：数据校验通过后，会生成模型实例对象，可直接通过.属性访问字段（如 obj.price），相比原生字典语法更简洁直观。

【例 2.3-6】基础 Pydantic 模型定义与校验示例

【程序代码】

```
from pydantic import BaseModel, Field, field_validator

# 自定义数据模型，必须继承 BaseModel 基类
class User(BaseModel):
    # 字段定义：字段名: 数据类型 = Field(描述、默认值、约束条件)
    name: str = Field(description="用户姓名，不可为空")
    age: int = Field(description="用户年龄，整数", gt=0, le=150)
    city: str = Field(default="未知", description="用户居住城市，无填写时默认未知")
    hobbies: list[str] = Field(description="个人爱好字符串列表")

    # 自定义字段校验规则（可选，用于强化特殊业务约束）
    @field_validator("name")
    def name_not_empty(cls, value):
        # 校验姓名不能为纯空白字符串
        if not value.strip():
            raise ValueError("姓名不能为空")
        return value

if __name__ == "__main__":
    # 场景 1：入参完全符合规范，正常实例化对象
    user1 = User(
        name="张三",
        age=25,
        city="北京",
        hobbies=["看书", "跑步"]
    )
    # 直接通过属性读取字段数据
    print(user1.name) # 输出：张三
    print(user1.age) # 输出：25
    # 模型转为字典，适配数据库存储、接口传参等场景
    print(user1.model_dump())

    print("="*50)

    # 场景 2：入参违反约束，自动抛出校验异常
    try:
        user2 = User(name="", age=0, hobbies=["打游戏"])
    except Exception as e:
        print(f"数据校验失败：{e}")
```

2、智能体 response_format 参数

`create_agent` 内置原生结构化输出能力，仅需配置 `response_format` 参数，即可统一约束模型输出格式。模型生成合规结构化数据后，框架会自动完成捕获、校验，最终存入智能体运行状态的 `structured_response` 字段中。

函数参数定义

```
def create_agent(
    response_format: ResponseFormat[ResponseT] | type[ResponseT] | dict[str, Any] | None = None
)
    ...
```

- 参数说明：`response_format` 为可选参数，用于配置智能体标准化输出格式
- 支持传入类型：Pydantic 模型类、标准 `ResponseFormat` 对象、JSON Schema 字典

3、轻量级 JSON 格式输出

若业务无需复杂的数据类型、字段合法性强校验，可采用轻量化 JSON 输出方案。该方案无需配置 `response_format` 参数，仅通过系统提示词严格约束输出规则，强制模型返回可直接解析的标准 JSON 字符串。

【例 2.3-7】轻量化 JSON 输出

【程序代码】

```
import json

# 系统提示词，严格约束 JSON 输出规范
SYSTEM_PROMPT = """
你是农业病虫害诊断助手，输出必须严格遵守以下规则：
1. 返回 JSON 包含固定 4 个字段：
    - crop: 作物名称（示例：小麦、水稻、苹果）
    - disease: 病虫害名称（示例：小麦条锈病、水稻稻飞虱）
    - risk_level: 风险等级，仅允许填写：低、中、高
    - suggestion: 简易防治建议，文字不超过 50 字
2. 仅输出纯 JSON 文本，禁止额外解释、说明、标题、换行注释；
3. 保证 JSON 语法标准、可被程序直接解析。
"""

# 组装智能体
agent = create_agent(
    model=llm,
    system_prompt=SYSTEM_PROMPT
)

if __name__ == "__main__":
    user_input = ["分析当前麦田出现的黄叶、锈斑症状，给出病虫害诊断结果"]
    result = agent.invoke({"messages": user_input})
    # 取出模型原始输出文本
    raw_text = result["messages"][-1].content
```

```
# 手动解析 JSON
try:
    diagnosis = json.loads(raw_text)
    print(f'作物: {diagnosis.get('crop')}')
    print(f'病虫害: {diagnosis.get('disease')}')
    print(f'风险等级: {diagnosis.get('risk_level')}')
    print(f'防治建议: {diagnosis.get('suggestion')}')
except Exception as e:
    print(f'JSON 文本解析失败: {str(e)}')
    print("模型原始输出内容: ", raw_text)
```

4、Pydantic 模型结构化输出

LangChain 结构化输出官方推荐使用 Pydantic 数据模型，该方案适配所有支持工具调用的主流大模型，覆盖绝大多数业务场景。

【例 2.3-8】基于 Pydantic 的智能体结构化输出

本案例采用 LangChain 官方推荐的 Pydantic 结构化输出方案，通过 `response_format` 参数绑定数据模型，框架自动约束大模型输出格式、自动完成数据校验，无需手动编写 JSON 约束提示词。

【程序代码】

```
from pydantic import BaseModel, Field

# 1、定义病虫害诊断结构化 Pydantic 模型
class DiseaseResult(BaseModel):
    """农业病虫害诊断结构化输出数据模型"""
    crop: str = Field(description="作物名称，示例：小麦、水稻、苹果")
    disease: str = Field(description="病虫害名称，示例：小麦条锈病、水稻稻飞虱")
    risk_level: str = Field(description="风险等级，仅允许：低、中、高", pattern="^(低|中|高)$")
    suggestion: str = Field(description="简易防治建议，文本长度不超过 50 字", max_length=50)

# 基础角色提示词，无需手动约束 JSON 格式，框架自动处理
SYSTEM_PROMPT = "你是专业农业病虫害诊断助手，根据作物症状给出标准化诊断结果"

# 组装智能体：response_format 直接传入 Pydantic 模型类
agent = create_agent(
    model=llm,
    system_prompt=SYSTEM_PROMPT,
    response_format=DiseaseResult
)
```

```
if __name__ == "__main__":
    user_input = ["分析当前麦田出现的黄叶、锈斑症状，给出病虫害诊断结果"]
    # 流式调用智能体，实时更新状态
    result = agent.invoke({"messages":user_input})
    # 直接获取 Pydantic 模型实例，支持.属性读取
    diagnosis = result["structured_response"]
    print(f'作物: {diagnosis.crop}')
    print(f'病虫害: {diagnosis.disease}')
    print(f'风险等级: {diagnosis.risk_level}')
    print(f'防治建议: {diagnosis.suggestion}')
```

【程序执行结果解读】

程序执行后 LangChain 底层自动使用模型定义向大模型下发格式要求，底层自动选择模型原生工具调用 / 格式输出策略，强制大模型输出合规结构化数据，校验完成后的结构化对象自动存入会话状态 structured_response，直接为 Pydantic 实例，直接读取属性，比字典更便捷。

LangChain 结构化输出能力依托 Pydantic 模型、JSON Schema 两种规范定义数据格式，实现无约束自然语言到大模型强类型结构化数据的自动化转换与合法性校验。

核心业务价值：

- 降低文本解析开发成本，无需手动编写复杂文本提取逻辑
- 统一输出格式，多轮对话、多次调用输出结果保持一致
- 自带数据校验，提前拦截不合规数据，减少下游程序异常
- 结构化数据可无缝对接数据库存储、后端 API、工具函数等下游业务系统

2.4 大模型异步与流式处理体系

流式传输是构建高交互、低等待 AI 农业应用的核心能力。LangChain 中所有 Runnable 组件（LLM、Chain、Agent）统一提供同步 / 异步、单次 / 批量、流式 5 套标准调用接口，同时支持 3 种流式输出模式，适配本地调试、后端高并发、前端 SSE 实时聊天、工具日志监控全场景。

2.4.1 五大标准调用接口

所有 Agent、LLM、LCEL 链路共享同一套标准化调用接口，分为同步、异步两大类，覆盖单次、批量、流式场景。

1、同步单次 invoke ()调用

同步阻塞执行，等待整条链路完整执行完毕后一次性返回完整结果，适合本地脚本、离线调试、单次问答。

【例 2.4-1】Agent 五大标准调用方式

【程序代码】

```
SYSTEM_PROMPT = """
你是一个农业专家，能够根据用户输入的农业问题给出对应的回答。
"""

agent = create_agent(model=llm,system_prompt=SYSTEM_PROMPT)
# 同步单次调用
if __name__ == "__main__":
    res = agent.invoke({"messages": "玉米每亩适宜种植密度? "})
    print(res["messages"][-1].content)
```

(2) 同步批量 batch () 调用

一次性传入多条用户提问，批量执行完整的生成流程，多条任务并行处理，适合批量问答、批量文档摘要、批量知识库检索场景。

```
# 批量同步执行
if __name__ == "__main__":
    batch_questions = [
        "小麦拔节期施肥方案",
        "玉米密植每亩多少株合适",
        "水稻稻田综合种养注意事项"
    ]
    # 将问题列表转换为字典列表
    batch_inputs = [{"messages": q} for q in batch_questions]
    # 执行批量调用
    batch_results = agent.batch(batch_inputs)
    # max_concurrency 控制并发请求数量，避免接口限流
    # batch_results = agent.batch(batch_inputs,config={"max_concurrency": 2})
    # 遍历打印结果
    for q, ans in zip(batch_questions, batch_results):
        print(f"【问题】 {q}")
        print(f"【回答】 {ans["messages"][-1].content}\n" + "-"*60)
```

(3) 异步单次 `ainvoke()` 调用

异步非阻塞执行完整链路，适配 FastAPI 后端、Web 服务高并发场景，不阻塞主线程，支持同时处理多条用户请求。

```
import asyncio
# 异步单次问答
async def single_ainvoke(input:dict):
    ans = await agent.ainvoke(input)
    print(ans["messages"][-1].content)

if __name__ == "__main__":
    question = "小麦抗旱拌种方法"
    input = {"messages": question}
    # Jupyter 内置支持直接执行 await 顶层协程，直接写 await 调用异步函数。
    await single_ainvoke(input)
    # .py 文件执行时，必须使用 asyncio.run() 函数执行异步程序。
    # asyncio.run(single_ainvoke(input))
```

(4) 异步流式 `astream()` 调用

异步逐块（分片）流式输出，打字机实时效果，对接前端 SSE 聊天窗口，边检索边分段返回文字，大幅降低用户等待感知。

如果希望直接获取 `AIMessageChunk`，可以在调用 `astream` 时指定 `stream_mode="messages"`：

```
async def stream_astream(input: dict):
    # 异步循环读取分片
    async for chunk in agent.astream(input, stream_mode="messages"):
        # 此时 chunk 通常是 (AIMessageChunk, metadata) 元组, chunk[0] 为 AIMessageChunk
        # 分片对象
        # end="": 打印后不换行，文字连续拼接
        # flush=True: 强制刷新输出缓冲区，文字立刻打印，实现打字机实时效果；
        print(chunk[0].content, end="", flush=True)

if __name__ == "__main__":
    question = "小麦拔节期钾肥施用注意事项"
    input = {"question": question}
    await stream_astream(input)
```

(5) 异步批量 `abatch()`

异步批量处理多条提问，高并发服务场景最优选择。

```
from typing import List
async def batch_abatch(questions: List[str]):
    results = await agent.abatch([{"messages": question} for question in questions])
    for question, result in zip(questions, results):
        print(f"【问题】： {question}")
        print(f"【回答】： \n{result["messages"][-1].content}")
```

```

if __name__ == "__main__":
    questions = [
        "小麦拌种抗旱技巧",
        "玉米滴灌施肥方案"
    ]
    await batch_abatch(questions)

```

在 LangChain 中，Chain、Agent、LLM 等核心组件的标准执行方法主要是五种调用方式，它们者在输出形式、类型、适用场景上存在本质区别，如表 2.4-1 所示：

表 2.4-1 五种调用方式的区别

调用方法	类型	核心适用场景	输出特点
invoke()	同步单次	本地调试、单次简单问答	完整文本一次性返回
batch()	同步批量	离线批量生成问答、批量摘要	同步等待全部任务完成，返回结果列表
ainvoke()	异步单次	Web 后端接口、高并发单用户请求	非阻塞，后台执行，一次性返回完整回答
astream()	异步流式	前端聊天实时打字效果	逐 Token 分片实时输出
abatch()	异步批量	后端批量高并发任务	异步并行批量处理多条提问

2.4.2 三类流式输出模式

流式传输是构建高响应 AI 应用的核心能力。借助分片实时输出，无需等待完整响应生成，即可向用户推送内容，大幅优化用户等待体验；LangChain 提供三类独立流式输出模式，分别适配实时打字、工具进度日志、Agent 执行流程监控三大场景。

1、LLM Token 流式输出

（1）核心原理

大模型生成文本时不会等全部内容生成完毕再统一返回，每生成一组 Token 就实时通过流传输给客户端，实现边生成、边传输、边展示的打字机交互效果。

LangChain 中分片数据载体为 AIMessageChunk，是完整 AI 消息 AIMessage 的分片子类；在流式模式下，迭代器返回元组（AIMessageChunk，元数据字典），通过 chunk[0].content 提取文本分片。

（2）关键约束

如需逐 Token 流式输出，调用 stream()/astream()时必须显式配置参数：

```
stream_mode="messages"。
```

该参数指定流式输出模式为模型 Token 分片，大模型每生成一小段文字就立刻返回，而非等待全文生成完毕一次性输出，实现前端 / 控制台打字机实时输出。

【例 2.4-2】LLM 流式输出

本案例实现两种 LLM 流式输出方案：同步 stream()、异步 astream()。采用 stream_mode="messages" 逐 Token 分片输出模型回答，模拟打字机实时打印效果。

```

# 1. 依赖导入
import asyncio
from langchain.agents import create_agent
from langchain_core.messages import SystemMessage, HumanMessage

# 2. 角色系统提示

```

```
SYSTEM_PROMPT = SystemMessage("你是一个农业专家，请为我答疑，回答时不需要添加额外内容。")
```

```
# 3.同步流式函数
```

```
def sync_stream_demo():
    agent = create_agent(model=llm, system_prompt=SYSTEM_PROMPT)
    human_message = HumanMessage("请告诉我如何预防桔子的黑点病？")
    messages = [human_message]
    # 同步遍历流式分片
    for chunk in agent.stream({"messages": messages}, stream_mode="messages"):
        print(chunk[0].content, end="", flush=True)
    print("\n")
```

```
# 4. 异步流式函数，async def: 标记为异步函数，内部可执行异步 IO 操作
```

```
async def async_stream_demo():
    agent = create_agent(model=llm, system_prompt=SYSTEM_PROMPT)
    human_message = HumanMessage("请告诉我如何预防水稻的稻瘟病？")
    messages = [human_message]
    # 异步生成器必须使用 async for 遍历
    async for chunk in agent.astream({"messages": messages}, stream_mode="messages"):
        print(chunk[0].content, end="", flush=True)
```

```
# 5. 运行
```

```
if __name__ == "__main__":
    print("===== 同步流式输出演示 =====")
    sync_stream_demo()

    print("===== 异步流式输出演示 =====")
    # 异步程序标准入口：
    # Jupyter 内置支持直接执行 await 顶层协程，直接写 await 调用异步函数。
    await async_stream_demo()
    # .py 文件执行时，必须使用 asyncio.run() 函数执行异步程序。
    # asyncio.run(async_stream_demo())
```

【程序执行结果解读】

本程序以流式流式逐 Token 输出，用户无需长时间等待完整回答，交互体验更好。双流模式方案覆盖测试与生产场景，同步适合本地调试，异步适配线上并发服务。

2、自定义更新流式

自定义更新流式用于在工具、节点执行过程中实时推送自定义日志、进度提示、中间业务数据（例如：已获取 10/100 条记录、接口请求日志）。

自定义更新流式需要配置参数：

```
stream_mode="custom"
```

数据推送规则：仅允许在 Agent 工具、节点的执行上下文中使用 `get_stream_writer()` 推送数据，推送内容需为字典格式的自定义数据流。

数据接收规则：程序可捕获工具执行过程中实时推送的自定义数据流，实现执行进度、

运行日志实时输出。

【例 2.4-3】自定义流式更新

本案例借助异步流式与自定义流实现实时打印工具执行日志。

```
import asyncio
import requests
from langchain.tools import tool
from langchain.agents import create_agent
from langgraph.config import get_stream_writer
from langchain_core.messages import HumanMessage

# 自定义获取谚语工具（内置实时日志推送）
@tool
def get_proverb() -> str:
    """
    获取谚语。
    返回: 谚语字符串
    """
    # 推送自定义日志流
    writer = get_stream_writer()
    try:
        writer({"type": "log", "content": "正在获取谚语..."})
        url = "https://cn.apihz.cn/api/zici/yanyu.php?id=10019433&key=fe034a161aeec2bf45f841b6024e898b"
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        writer({"type": "log", "content": "获取谚语已完成"})
        data = response.json()
        if data.get("code") != 200:
            writer({"type": "log", "content": f"获取谚语失败: {data.get('msg')}"})
            return f"获取笑话失败: {data.get('msg')}"
        else:
            writer({"type": "log", "content": "正在整理谚语内容..."})
            content = data.get("content")
            writer({"type": "result", "content": content})
            writer({"type": "log", "content": "谚语生成完成"})
            return data
    except Exception as e:
        writer({"type": "log", "content": f"获取谚语失败: {str(e)}"})
        return f"获取谚语失败: {str(e)}"

# 异步封装自定义流读取逻辑
async def stream_proverb_query():
    """
    异步执行 Agent，流式输出日志，最后汇总输出最终结果
    """
```

```

"""
SYSTEM_PROMPT = """你是一个经验丰富的农民：
1.使用下面的工具获取农林相关谚语：
- get_jproverb： 获取谚语
2. 禁止编造内容，基于工具返回谚语优化为贴合农业、农林主题，篇幅≤100 字
3. 直接输出优化后的谚语文本
"""

agent = create_agent(
    model=llm,
    tools=[get_proverb],
    system_prompt=SYSTEM_PROMPT
)
messages = [HumanMessage(content="请给告诉我一个谚语")]
async for chunk in agent.astream({"messages": messages}, stream_mode="custom"):
    if not chunk: # 忽略空数据
        continue
    if isinstance(chunk, dict):
        chunk_type = chunk.get("type")
        content = chunk.get("content", "")
        if chunk_type == "log":
            print(f"[日志] {content}")
        elif chunk_type == "result":
            print(f"[结果] {content}")

if __name__ == "__main__":
    await stream_proverb_query()
    # asyncio.run(stream_proverb_query())

```

【程序执行结果解读】

程序运行时会通过自定义流式实时打印接口请求、数据整理、执行失败等各类工具运行日志，并单独输出接口返回的原始谚语内容。

3、Agent 执行进度流式

Agent 进度流式会在每一个图节点执行完成后推送增量状态更新，完整监控 Agent 全流程执行步骤，可清晰区分三类节点行为：

- LLM 模型节点：输出携带工具调用指令的 AI 消息（模型决策调用工具）；
- 工具节点：输出工具执行返回结果；
- 最终 LLM 节点：输出整合工具结果后的最终回答文本。

关键约束：

监控 Agent 分步执行进度，必须配置参数：

```
stream_mode="updates"
```

【例 2.4-4】查看 Agent 执行进度

本案例借助 stream_mode="updates" 流式模式，分步监控智能体完整执行流程，打印每一步模型 / 工具节点名称与消息内容块，实现 Agent 全流程调试可视化。

```
async def stream_agent_progress():
```

```

"""
异步执行 Agent，流式输出日志，最后汇总输出最终结果
"""

SYSTEM_PROMPT = """你是一个经验丰富的农民：
1.使用下面的工具获取农林相关谚语：
- get_jproverb：获取谚语
2. 禁止编造内容，基于工具返回谚语优化为贴合农业、农林主题，篇幅≤100 字
3. 直接输出优化后的谚语文本
"""

agent = create_agent(
    model=llm,
    tools=[get_proverb],
    system_prompt=SYSTEM_PROMPT
)

messages = [HumanMessage(content="请给告诉我一个谚语")]
# stream_mode="updates" 获取每一步节点增量更新
async for chunk in agent.astream({"messages": messages}, stream_mode="updates"):
    if not chunk:
        continue
    # 遍历每一个执行节点（model / tools）
    for step, data in chunk.items():
        print(f"step: {step}")
        # 读取当前节点最新消息内容块
        last_msg = data["messages"][-1]
        print(f"content: {last_msg.content_blocks}")
        print("-" * 60)
if __name__ == "__main__":
    await stream_agent_progress()
    # asyncio.run(stream_agent_progress())

```

【程序执行结果解读】

程序异步流式遍历智能体每一步模型、工具执行节点，实时打印节点名称与消息内容块，完整可视化展示智能体从模型决策、工具调用到生成最终结果的全执行流程。

LangGraph 通过 stream()/astream()搭配三种 stream_mode 实现全场景流式输出，主要优势包括：

- 交互体验优化：LLM Token 流式实现打字机实时输出，消除用户空白等待；
- 流程可视化：自定义流推送工具执行日志、进度；updates 模式完整监控 Agent 模型 / 工具分步执行流程；
- 内存资源优化：分片增量输出，无需缓存完整长文本，降低内存占用；
- 数据可信可控：全流程中间数据实时输出，便于调试、日志埋点、前端实时渲染。

三种流式模式选型参考：

- messages：聊天对话、实时打字机展示、逐 Token 输出文本
- custom：工具执行日志、任务进度条、中间业务数据推送

- updates: 调试监控 Agent 执行链路、查看模型工具分步调用过程

教材未出版，禁止传播，版权所有：柯善斌

2.5 表达式语言

2.5.1 表达式语言基础

1、核心概念

LangChain 中的表达式语言 LCEL (LangChain Expression Language) 是 LangChain 核心的链式调用语法 (Chain)。它是一种组件组合范式，通过管道符 “|” 将多个独立的功能组件 (如 Prompt、LLM、Parser、Tool、Memory 等) 串联成一个可执行的“流水线”，让数据自动从左到右流经每个组件完成处理，无需手动分步传递中间结果。

2、链式调用

可以把链式调用理解为：你要做一道菜（完成一个 AI 任务），需要经历“备料→烹饪→装盘”三个步骤，链式调用就是把这三个步骤固定成一条自动化生产线，你只需放入原材料（输入数据），就能直接拿到做好的菜（最终结果）。

在 LangChain 中：

- 组件 = 生产步骤（备料 / 烹饪 / 装盘）；
- 管道符 | = 生产线的传送带；
- 输入数据 = 原材料；
- 最终结果 = 做好的菜。

1) 通用格式：组件 1 | 组件 2 | 组件 3 | ...

```
chain = 组件 A | 组件 B | 组件 C
```

用 | 串联组件时，必须保证前一个组件的输出就是后一个组件的输入。

2) 执行调用

(1) 同步调用

```
content= chain.invoke(输入数据)
print(content)
```

(2) 流式输出

```
for chunk in chain.stream(输入数据):
    print(chunk,end="",flush=True)
```

3、解释器

LangChain 解析器的核心作用是将大语言模型 (LLM) 返回的非结构化文本，转换成结构化的数据（如字典、列表、Pydantic 模型等），让你能直接用代码处理模型输出，而不用手动写字符串处理逻辑。

1) 简单解析器

StringOutputParser 是最基础的解析器，仅将模型输出转为纯字符串，适合不需要结构化的场景。

2) 结构化解析器

JsonOutputParser 是将模型输出转为 JSON 格式 (Python 字典)，适合需要提取多字段信息的场景（如提取作物名称、病虫害、风险等级）。

3) 强类型解析器、

PydanticOutputParser 是基于 Pydantic 模型定义输出结构，强制模型输出符合格式的内容，还能自动校验字段类型，适合生产环境。

使用 `JsonOutputParse` 或 `PydanticOutputParser` 类型转换时，必须在 `Prompt` 模板中指定 `get_format_instructions()`。它的核心作用是自动生成清晰、标准化的格式说明文本，并将其传入提示词中，告诉大语言模型（LLM）必须按照你定义的 `Pydantic` 模型结构来输出内容。

```
prompt = prompt.partial(format_instructions=parser.get_format_instructions())
```

同时在系统提示词中说明“格式要求：{format_instructions}”。占位符 {format_instructions} 是将 `get_format_instructions()` 生成的标准化格式指令，嵌入到发给大语言模型的提示词中，让模型明确知道“必须按照这个格式输出”，是连接 `Pydantic` 模型和 LLM 输出的关键桥梁。

最基础的调用是“提示词模板 + LLM + 输出解析器”的链式组合，应用没有调用工具的智能体中。

【例 2.5-1】简单的链式调用

在示例中，让 LLM 回答专业性问题，程序执行流程为：

第一步：构建提示词，接收用户输入，生成完整的提示词；

第二步：把提示词传给 LLM，调用模型生成响应；

第三步：把 LLM 响应传给字符串解析器，解析为纯字符串。

这个步骤概括起来就是输入数据（Prompt）| 调用模型（LLM）| 解析结果（Parser），这三个步骤串联成一个可执行的“流水线”，让数据自动从左到右流经每个组件完成处理。

【程序代码】

```
from langchain_core.output_parsers import StrOutputParser
from langchain_core.prompts import ChatPromptTemplate
# 1. 构建提示词
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是一个农业助手，请简单解答我的问题，回答时不需要添加额外内容。"),
    ("human", "{question}")
])
# 2. 选取结果解析器
parser = StrOutputParser()
# 3. 构建调用链
chain = prompt | llm | parser
# 4. 链式调用
if __name__ == "__main__":
    question = {"question": "冬天下雪，会给来年作物生长带来哪些方面的好处?"}
    # 5.1 同步调用
    # content = chain.invoke(question)
    # print(content)
    # 5.2 异步调用
    for chunk in chain.stream(question):
        print(chunk, end="", flush=True)
```

2.5.2 Runnable 组件

`Runnable` 是 `LangChain` 框架的核心抽象，简单来说，所有能被管道（|）串联的组件，

本质上都是 Runnable。它定义了统一的接口（invoke()/stream()/batch()），让不同功能的组件（提示词、模型、工具、解析器等）能像“乐高积木”一样灵活组合，这也是“|”语法构建调用链的底层基础。

1、Runnable 核心作用

1) 统一接口：无论是什么组件（模型、提示词、自定义函数），只要实现 Runnable 接口，就拥有 invoke()（单次调用）、stream()（流式输出）、batch()（批量调用）方法；

2) 管道兼容：只有 Runnable 组件才能用“|”语法串联，保证“前一个输出就是后一个输入”的流畅衔接；

3) 状态隔离：每个 Runnable 都是独立的，调用时互不干扰，便于调试和复用。

2、常用 Runnable 组件类型

LangChain 内置了大量开箱即用的 Runnable 组件，主要包括以下几类：

1) 基础 Runnable 组件

这些组件是内置组件，无需手动封装，实现了 Runnable 接口。即直接用“|”串联的都是 Runnable。

- 提示词模板：BaseChatPromptTemplate
- 聊天模型：BaseChatOpenAI
- 解析器：BaseOutputParser

2) 自定义 Runnable 组件

(1) RunnableLambda

这是最常用的自定义方式。将普通函数封装为 Runnable，适配管道语法。

【例 2.5-2】用 RunnableLambda 将普通函数封装为 Runnable

【程序代码】

```
from langchain_core.runnables import RunnableLambda
# 普通函数（输入数字，返回平方）
def square(x: int) -> int:
    return x * x
# 封装为 Runnable
square_runnable = RunnableLambda(square)
# 调用（支持 invoke/管道）
print(square_runnable.invoke(5)) # 输出：25
# 串联到管道中
chain = RunnableLambda(lambda x: x + 1) | square_runnable
print(chain.invoke(4)) # (4+1)2=25 → 输出：25
```

(2) RunnableSequence

如果需要更灵活的组合，可以用 RunnableSequence。其实“|”语法本质上就是创建 RunnableSequence。

管道输出可以使用 RunnableSequence：

```
# 方式 1：用|创建
chain1 = prompt | llm | parser
# 方式 2：显式创建（等价于方式 1）
chain2 = RunnableSequence(first=prompt, middle=[ llm], last=parser)
```

(3) RunnableParallel

并行即同时执行多个 Runnable，每个分支返回结果以字典形式封装。

(4) RunnableBranch

根据条件选择不同的分支 Runnable 执行，类似 if-else 语句。

【例 2.5-3】并行和分支 Runnable

【程序代码】

```
from langchain_core.runnables import RunnableParallel, RunnableBranch
# 定义两个 Runnable
r1 = RunnableLambda(lambda x: x + 1)
r2 = RunnableLambda(lambda x: x * 2)
# 并行执行
parallel_chain = RunnableParallel(add_one=r1, multiply_two=r2)
result = parallel_chain.invoke(5)
print(result) # 输出: {"add_one": 6, "multiply_two": 10}
# 分支执行
branch_chain = RunnableBranch(
    (lambda x: x > 0, r1),
    (lambda x: x < 0, r2),
    # 默认分支
    r1
)
result = branch_chain.invoke(5)
print(result) # 输出 6
result = branch_chain.invoke(-5)
print(result) # 输出 -10
```

2.5.3 LCEL 典型应用场景

场景 1：带自定义工具的线性链式调用

Agent 原生循环逻辑不能直接用管道串联，需通过 RunnableLambda 封装工具函数适配 LCEL 链路。

【例 2.5.4】图像生成 + 保存完整工具链

功能：根据农业病害描述生成图像，自动下载保存本地，使用 Pydantic 强类型结构化输出。

智能体是循环调用的：提示词—模型—模型输出（决策）—工具调用—模型（结果）。所以带工具的智能体链式调用遵循以下步骤：

第一步定义基础链：提示词模板 | LLM | 输出解析器

第二步定义式具链：基础链 | 工具链 1 | 工具链 2 | ……

【程序代码】

```
import os, time, requests
from pydantic import BaseModel, Field, field_validator
from langchain_core.prompts import ChatPromptTemplate
from openai import OpenAI
from langchain_core.output_parsers import PydanticOutputParser
```

```

from langchain_core.runnables import RunnableLambda

# 1. 定义图像输入输出强类型模型
class Images(BaseModel):
    prompt: str = Field(description="图片生成提示词，农业领域相关",default=None)
    size: str = Field(description="图片分辨率",default="1024x1024")
    n: int = Field(description="图片数量", gt=1, lt=4,default=1)
    urls: list[str] = Field(description="图片远程 URL 列表或本地保存路径列表",default=None)
    timestamp: float = Field(description="图片生成时间戳",default=time.time())
    @field_validator("size")
    def validate_size(cls, value):
        valid_sizes = ["1024x1024", "960x1280", "768x1024", "720x1440", "720x1280"]
        if value not in valid_sizes:
            raise ValueError(f"分辨率无效，可选：{valid_sizes}")
        return value

# 2. 封装工具函数
# 2.1 封装图像生成工具函数
def generate_image(params:Images) -> Images:
    """
    调用图像生成 API，返回图像 URL 列表
    参数：
    params: Images 图片参数
    返回：
    Images 图片输出模型
    """
    client = OpenAI(api_key=os.getenv("OPENAI_API_KEY"),
base_url=os.getenv("BASE_URL"))
    response = client.images.generate(
        model="Kwai-Kolors/Kolors",
        prompt=params.prompt,
        size=params.size,
        n=params.n
    )
    return Images(
        prompt=params.prompt,
        size=params.size,
        n=params.n,
        urls=[item["url"] for item in response.images],
        timestamp=response.created
    )

# 2.2 封装图片保存工具函数

```

```
def save_image(images: Images) -> Images:
    """
    下载远程图片保存至本地，替换 URL 为本地文件路径
    参数：
    images: Images 图片输入模型
    返回：
    Images 图片输出模型
    """
    paths = []
    for i, url in enumerate(images.urls):
        file_name = f"{images.timestamp}_{i}.png"
        file_path = f"./{file_name}"
        img_response = requests.get(url)
        img_response.raise_for_status()
        with open(file_path, "wb+") as f:
            f.write(img_response.content)
        paths.append(file_path)
    return Images(
        prompt=images.prompt,
        size=images.size,
        n=images.n,
        urls=paths,
        timestamp=images.timestamp
    )

# 3. 构建基础链路
# 3.1 构建提示词
PROMPT_TEMPLATE = """
请根据以下用户描述，生成一个详细、专业的农业病害图片生成提示词：
{user_input}

要求：
1. 包含病害特征的详细描述
2. 描述叶片颜色、形状变化
3. 增加光照、清晰度等要求
4. 只输出优化后的提示词，不要其他内容
5. 输出格式： {format_instructions}
"""

prompt = ChatPromptTemplate.from_template(PROMPT_TEMPLATE)
parser = PydanticOutputParser(pydantic_object=Images)
prompt = prompt.partial(format_instructions=parser.get_format_instructions())
# 3.2 构建基础链路
base_chain = prompt | llm | parser
```

4. 构建解析链路

4.1 封装工具为 Runnable

```
generate_image_runnable = RunnableLambda(generate_image)
```

```
save_image_runnable = RunnableLambda(save_image)
```

4.2 构建完整解析链路

```
full_chain = base_chain | generate_image_runnable | save_image_runnable
```

5. 链路调用测试

```
if __name__ == "__main__":
```

```
    user_input = ""
```

描述：水稻稻叶上出现黄色小斑点，逐渐扩大并融合成斑块，黄化区域伴有霉层和小黑点等病菌特征。

需求：绘制 2 张 720x1280 病害图片，要求清晰。""

```
    try:
```

```
        result = full_chain.invoke(user_input)
```

```
        print(f"【结果】：\n{result}")
```

```
    except Exception as e:
```

```
        print(f"调用失败：{str(e)}")
```

场景 2：并行多任务链路

【例 2.5-5】图像生成 + 病虫害防治方案同时执行

基于上例图像生成链路，新增病虫害诊断防治链，使用 `RunnableParallel` 并发执行两个任务，同时返回图片本地路径与病害防治方案。

①添加一条病虫害防治方案调用链，并构建并行链：

```
from langchain_core.runnables import RunnableParallel
```

```
# 构建防治链（生成病虫害防治方案）
```

```
# 提示词
```

```
diagnosis_prompt = ChatPromptTemplate.from_messages([
```

```
    ("system", "你是一个农业病虫害专家，请根据输入的病害描述，给出简单的病虫害防治方案"),
```

```
    ("human", "{user_input}")
```

```
])
```

```
# 输出格式
```

```
diagnosis_parse = StrOutputParser()
```

```
# 调用链
```

```
diagnosis_chain = diagnosis_prompt | llm | diagnosis_parse
```

```
# 构建并行链
```

```
parallel_chain = RunnableParallel(images=full_chain, diagnosis=diagnosis_chain)
```

②调用并行链

```
if __name__ == "__main__":
```

```
    user_input = ""
```

描述：水稻稻叶上出现黄色小斑点，逐渐扩大并融合成斑块，黄化区域伴有霉层和小黑点等病菌特征。

需求：绘制 2 张 720x1280 病害图片，要求清晰。"""

```
try:
    # 执行管道调用（一行代码完成所有流程）
    result = parallel_chain.invoke(user_input)
    print(f【生成图片】：\n{result["images"].urls}\n【防治方案】：\n{result["diagnosis"]}\n')
except Exception as e:
    print(f'调用失败：{str(e)}')
```

场景 3：条件分支链路

【例 2.5-6】根据角色切换任务

使用 RunnableBranch 实现路由判断：输入角色为绘图执行图像生成链路；角色为防治执行病害诊断链路。

①创建分支链：

```
from langchain_core.runnables import RunnableBranch
# 构建分支链
branch_chain = RunnableBranch(
    (lambda x: "防治" in x["role"], diagnosis_chain),
    (lambda x: "绘图" in x["role"], full_chain),
    full_chain
)
```

②调用分支链

```
if __name__ == "__main__":
    user_input = """
        描述：水稻稻叶上出现黄色小斑点，逐渐扩大并融合成斑块，黄化区域伴有霉层和小黑点等病菌特征。
        需求：绘制 2 张 720x1280 病害图片，要求清晰。"""
    try:
        # 执行管道调用（一行代码完成所有流程）
        result = branch_chain.invoke({"user_input": user_input, "role": "绘图"})
        print(result.urls)
        for chunk in branch_chain.stream({"user_input": user_input, "role": "防治"}):
            print(chunk, end="", flush=True)
    except Exception as e:
        print(f'调用失败：{str(e)}')
```

场景 4：带记忆多轮对话链路

基础记忆组件 BaseCheckpointSaver 不实现 Runnable 接口，需使用 RunnableWithMessageHistory 包装基础对话链，自动管理 Redis 和内存会话上下文，实现多轮问答记忆留存。RunnableWithMessageHistory 的参数：

- runnable：要绑定记忆的“基础执行链”（可以是 prompt+llm、自定义 chain 等）任意实现 Runnable 接口的对象。
- get_session_history：记忆获取函数：输入 session_id，返回对应的聊天记忆实例（Redis / 内存 / 数据库）。函数必须满足：

- 入参: session_id (字符串)
- 返回: BaseChatMessageHistory 子类实例 (如 RedisChatMessageHistory/ChatMessageHistory)
- input_messages_key: prompt 中的用户占位符 (可选)

【例 2.5-7】Redis 持久化记忆对话链

【程序代码】

```
import os
from langchain_community.chat_message_histories import RedisChatMessageHistory
from langchain_core.chat_history import BaseChatMessageHistory
from langchain_core.prompts import ChatPromptTemplate
from langchain_core.runnables import RunnableWithMessageHistory
from langchain_core.output_parsers import StrOutputParser

# 1. 基础问答链
prompt = ChatPromptTemplate.from_messages([
    ("system", "你是农业技术助手，请用简洁的语言专业解答种植问题"),
    ("human", "{question}")
])

parser = StrOutputParser()
chain = prompt | llm | parser

# 2. Redis 会话记忆获取函数
def get_redis_history(session_id:str) -> BaseChatMessageHistory:
    chat_history = RedisChatMessageHistory(
        session_id=session_id,
        url="redis://:123456@192.168.100.101:6379"
    )
    return chat_history

# 3. 绑定记忆到基础链路
chat = RunnableWithMessageHistory(
    runnable=chain,
    get_session_history=get_redis_history,
    input_messages_key="question"
)

# 4. 多轮对话测试，自动携带历史上下文
if __name__ == '__main__':
    config = {"configurable": {"session_id": "test"}}
    # 第一轮提问
    print("=====第一轮提问=====\n")
    for chunk in chat.stream(input={"question": "推荐南方冬季露天种植的青菜品种"},
config=config):
```

```
print(chunk,end="",flush=True)

# 第二轮自动记忆上文
print("\n=====第二轮提问=====\n")
for chunk in chat.stream(input={"question": "这些青菜中哪种营养价值更高"},
config=config):
    print(chunk,end="",flush=True)
# 第三轮延续上下文
print("\n=====第三轮提问=====\n")
for chunk in chat.stream(input={"question": "高营养青菜最佳播种时间"},
config=config):
    print(chunk,end="",flush=True)
```

LCEL 是 LangChain 核心的声明式表达式语言，以管道符“|”为核心语法将提示词、模型、工具、解析器等组件串联成可复用的调用链。通过统一的组件接口和极简的管道语法，实现组件间输入输出的无缝衔接，大幅降低链式调用的开发复杂度，同时保证调用链的可组合、可测试、可扩展，且天然支持记忆、流式输出等高级能力。

2.6 综合案例

2.6.1 系统功能概述

本项目搭建一套面向农业领域的智能问答原型系统，支持多轮对话记忆、私有知识库优先查询、通用知识兜底回答。前端效果图如 2.6-1 所示：



图 6.2-1 农业领域的智能问答原型系统效果图

1、核心业务流程

用户输入农业问题 — 服务接收请求 — 智能体优先检索私有农业知识库 — 无匹配则调用通用农业知识工具 — 大模型结果通过 SSE 实时流式推送前端 — 前端逐字符打字机效果展示回答。

2、主要功能模块

(1) **FastAPI 流式接口服务**：基于 SSE 协议提供长连接流式问答接口，支持跨域访问、异常捕获、会话隔离；

(2) **LangChain 农业智能体**：内置双工具调度策略，强制优先使用私有知识库，搭配大模型实现工具调用；基于内存实现多轮对话上下文记忆；

(3) **前端交互页面**：原生 HTML+JS 实现聊天界面，支持快捷提问、回车发送、请求中断、打字机动画，本地存储会话 ID 区分用户对话。

3、设计优势

- (1) 全异步架构：异步接口搭配智能体异步流式输出，资源利用率高，响应速度快；
- (2) 成熟流式交互：基于标准 SSE 实现打字机效果，交互体验接近主流大模型对话产品；
- (3) 灵活工具调度策略：固化业务规则 —私有内部知识库优先，通用知识兜底，贴合企业内部资料优先使用场景；
- (4) 会话隔离机制：通过 session_id 区分不同用户对话，支持多轮上下文连续问答；

4、现存不足与优化方案

(1) 知识库模块

现状：私有知识库采用硬编码字典存储，仅简单字符串匹配，无法承载大量文档，不支持 PDF、文档文件导入。

优化方案：接入 RAG 检索增强生成架构，实现文档加载、文本分割、Embedding 向量化、向量数据库相似度检索，替代现有简易匹配逻辑。

(2) 会话记忆

现状：基于内存存储对话记录，服务重启数据丢失，不支持多实例分布式部署。

优化方案：替换为 Redis Checkpointer，实现对话持久化，支持集群部署。

(4) 流式接口设计

现状：SSE 采用 GET 请求，长问题存在 URL 长度限制；后端添加人为延时模拟打字效果，占用服务资源。

优化方案：改用 POST 传输提问内容；删除后端 sleep 延时，打字动画逻辑统一交由前端控制。

2.6.2 编码实现

1、LangChain 农业智能体

代码内置双工具调度策略，强制优先使用私有知识库，搭配大模型实现工具调用；基于内存实现多轮对话上下文记忆。文件名: agri_agent.py:

```
# 文件名: agri_agent.py
import os
import asyncio
from dotenv import load_dotenv
from langchain_openai import ChatOpenAI
from langchain.tools import tool
from langchain.agents import create_agent
from langgraph.checkpoint.memory import InMemorySaver
from langchain_core.messages import HumanMessage, ToolMessage, AIMessage
from pydantic import BaseModel, Field

# 1. 加载环境变量 & 初始化大模型
load_dotenv(override=True)
llm = ChatOpenAI(
    base_url=os.getenv("BASE_URL"),
    openai_api_key=os.getenv("OPENAI_API_KEY"),
    model=os.getenv("CHAT_MODEL"),
```

```

        temperature=0.1,
        streaming=True # 启用流式
    )

# 2. 自定义双工具
@tool
def query_private_agri_db(question: str) -> str:
    """
    【优先级最高】查询私有农业知识库
    """
    private_knowledge_base = {
        "水稻稻瘟病防治": "发病初期喷施三环唑，田间排水降湿，避免偏施氮肥；分蘖期重点巡查叶瘟，抽穗期防穗颈瘟。",
        "赣南脐橙施肥方案": "春季萌芽肥高氮，膨果期氮磷钾均衡，采果后施有机肥恢复树势，忌夏季单一尿素。",
        "小麦蚜虫特效药": "吡虫啉、噻虫嗪，清晨施药，重点喷洒叶片背面，连片农田统一防治。",
        "大棚黄瓜霜霉病": "控温 22-26℃，降低棚内湿度，药剂可选霜霉威盐酸盐，7 天一次轮换用药。"
    }
    match_content = ""
    for key, val in private_knowledge_base.items():
        if key[:4] in question or question[:4] in key:
            match_content = val
            break
    if match_content:
        return f"【私有知识库数据】 {match_content}"
    else:
        return "无匹配数据"

@tool
def general_agri_knowledge(question: str) -> str:
    """
    兜底工具：私有知识库无匹配内容时调用
    """
    return f"【通用农业常识】请基于全国主流种植技术回答问题：{question}，内容客观、无编造，区分南北方种植差异。"

# 3. 系统提示词
SYSTEM_PROMPT = """
你是专业农业知识问答助手，严格遵守工具调用顺序规则：
1. 用户提问后，必须先调用 query_private_agri_db 私有知识库工具
2. 若私有工具返回无匹配数据：再调用 general_agri_knowledge 通用工具

```

仅基于工具返回数据或历史记忆回答，禁止编造额外信息，禁止 AI 幻觉！

输出格式：简洁专业的农业解答，包含必要的小贴士。

"""

4. 组装带记忆农业智能体

```
checkpointer = InMemorySaver()
agri_agent = create_agent(
    model=llm,
    tools=[query_private_agri_db, general_agri_knowledge],
    system_prompt=SYSTEM_PROMPT,
    checkpointer=checkpointer
)
```

5. 流式入口

```
async def agri_stream_chat(user_question: str, session_id: str = "agri_chat_001"):
    config = {"configurable": {"thread_id": session_id}}
    input_msg = {"messages": [HumanMessage(content=user_question)]}
```

用于追踪状态

```
full_answer = ""
source = "history_know"
tool_used = False
```

```
async for chunk in agri_agent.astream(
    input_msg,
    config=config,
    stream_mode="messages"
):
    try:
        # 处理元组格式 (message, metadata)
        if isinstance(chunk, tuple) and len(chunk) >= 1:
            msg = chunk[0]
            metadata = chunk[1] if len(chunk) > 1 else {}
```

捕获工具返回消息

```
if isinstance(msg, ToolMessage) and msg.content:
    content = msg.content
    print(f"工具返回: {content[:100]}...")
```

判断来源

```
if "【私有知识库数据】" in content:
    source = "private_db"
    # 提取纯答案
    answer = content.replace("【私有知识库数据】", "").strip()
```

```

        full_answer = answer
        tool_used = True
        # 直接发送工具结果
        yield {
            "type": "tool_result",
            "content": content
        }
    elif "【通用农业常识】" in content:
        source = "general_know"
        tool_used = True
        yield {
            "type": "tool_result",
            "content": content
        }
    elif "无匹配数据" in content:
        # 私有库无匹配，继续
        yield {
            "type": "tool_result",
            "content": content
        }
    continue

# 捕获 AI 流式输出
if isinstance(msg, AIMessage) and msg.content:
    content = msg.content
    if not tool_used:
        source = "history_know"
    if content.strip():
        # 逐字输出
        for char in content:
            yield {
                "type": "text_chunk",
                "chunk": char
            }
            await asyncio.sleep(0.02) # 微小延迟，让前端有打字效果
        full_answer += content

# 检查 metadata 中是否包含完整响应
if isinstance(metadata, dict) and "structured_response" in metadata:
    struct_data = metadata["structured_response"]
    if hasattr(struct_data, "model_dump"):
        struct_data = struct_data.model_dump()

# 发送完成事件，包含完整数据
yield {

```

```

        "type": "complete",
        "data": {
            "source": struct_data.get("source", source),
            "question": user_question,
            "answer": struct_data.get("answer", full_answer),
            "tip": struct_data.get("tip", "以上信息仅供参考")
        }
    }
    return

except Exception as e:
    print(f"处理 chunk 出错: {e}")
    import traceback
    traceback.print_exc()
    continue

# 如果没有收到 complete 事件，发送一个默认的
if full_answer:
    yield {
        "type": "complete",
        "data": {
            "source": source,
            "question": user_question,
            "answer": full_answer,
            "tip": "以上信息仅供参考，具体种植请结合当地实际情况。"
        }
    }
else:
    # 如果什么都没有，发送错误
    yield {
        "type": "error",
        "chunk": "未能获取到回答内容"
    }

```

2、FastAPI 流式接口服务

代码基于 SSE 协议提供长连接流式问答接口，支持跨域访问、异常捕获、会话隔离。
文件名: main.py:

```

# 文件名: main.py
import asyncio
import json
from fastapi import FastAPI, Request
from fastapi.responses import StreamingResponse
from agri_agent import agri_stream_chat
from fastapi.middleware.cors import CORSMiddleware
import logging

```



```

# 配置日志
logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

app = FastAPI(title="农业问答流式接口")

# 允许跨域
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

async def stream_generator(question: str, session_id: str):
    """流式生成器"""
    try:
        logger.info(f'开始处理问题: {question}')
        async for item in agri_stream_chat(question, session_id):
            # 序列化为标准 JSON 字符串
            json_str = json.dumps(item, ensure_ascii=False)
            yield f"data: {json_str}\n\n"
            logger.info(f'发送: {item.get('type', 'unknown')}')
    except Exception as e:
        logger.error(f'流式生成器错误: {e}')
        import traceback
        traceback.print_exc()
        error_data = {
            "type": "error",
            "chunk": f'处理失败: {str(e)}'
        }
        yield f"data: {json.dumps(error_data, ensure_ascii=False)}\n\n"

@app.get("/agri/stream")
async def agri_stream(request: Request):
    """流式问答接口"""
    question = request.query_params.get("question", "").strip()
    session_id = request.query_params.get("session_id", "default_session")

    if not question:
        return {"code": 400, "msg": "请输入有效的问题"}

```

```

logger.info(f"收到请求: question={question[:50]}...")

generator = stream_generator(question, session_id)
return StreamingResponse(
    generator,
    headers={
        "Content-Type": "text/event-stream",
        "Cache-Control": "no-cache",
        "Connection": "keep-alive",
        "Access-Control-Allow-Origin": "*",
    }
)

@app.get("/test")
async def test():
    """测试端点"""
    return {"status": "ok", "message": "服务运行正常"}

if __name__ == "__main__":
    import uvicorn
    uvicorn.run("main:app", host="0.0.0.0", port=5000, reload=True)

```

3、前端交互页面

原生 HTML+JS 实现聊天界面，支持快捷提问、回车发送、请求中断、打字机动画，本地存储会话 ID 区分用户对话：

详见文件：agri_assistant.html

本系统是一套可快速演示验证的农业领域 AI 问答最小可行原型，完整打通异步后端智能体、SSE 流式传输、前端打字机交互全链路，验证了“大模型智能体 + 工具调度 + 流式对话”技术方案可行性，能够满足场景原型展示需求。

当前版本仅适用于演示、实训验证，知识库未采用 RAG 架构，缺乏生产级高可用设计，不能直接投入正式业务使用。

后续迭代核心方向：落地 RAG 向量检索系统、重构智能体 workflow、持久化会话记忆、完善前后端健壮性，完成工程化改造，支撑真实业务场景落地。

教材未出版，禁止传播，版权所有：柯善斌

本章小结

本章围绕 LangChain 智能体的高级特性展开全面讲解，构建了从基础增强到进阶开发的完整知识体系，旨在帮助开发者突破基础使用局限，打造功能更强大、适配性更广、体验更优异的智能体应用。

(1) 在流式处理部分，核心聚焦于提升智能体的实时交互能力。通过对比 `invoke()` 与 `stream()/astream()` 两种执行方法的差异，明确了流式处理在长文本生成、高并发场景中的独特优势——即通过逐块返回 `AIMessageChunk` 分块内容，实现“边生成、边传输、边展示”的效果，有效降低用户等待感。同时，详细介绍了 LLM Tokens 流式输出、自定义进度更新、Agent 执行步骤跟踪三种核心流式模式，分别对应 `stream_mode=“messages | custom | updates”` 三种参数配置，覆盖了从基础打字机效果到工具执行日志推送、调用流程可视化的全场景需求，为实时交互类应用（如聊天机器人、任务进度监控系统）提供了完整解决方案。

(2) 中间件部分深入剖析了智能体的扩展机制，揭示了其“可插拔执行钩子”的本质。通过拆解智能体的全生命周期（启动、模型准备、核心执行、收尾），明确了中间件在各关键节点的拦截逻辑。重点讲解了 Runtime 运行时的核心资源（上下文、存储、流写入器），区分了瞬态与持久两种上下文类型的适用场景，同时详细介绍了 `@before_model` 等常用生命周期钩子函数的用法，结合日志记录、参数校验、异常重试等实战案例，展现了中间件在不侵入核心逻辑的前提下，对智能体功能进行灵活扩展的强大能力，为智能体的稳定性、可监控性提供了关键支撑。

(3) 高级智能体构建模块则聚焦于突破传统智能体的功能边界。动态系统提示通过中间件实现了根据用户角色、任务场景实时调整系统指令，让智能体响应更具针对性；多模态消息支持文本、图像、音频等多种数据类型输入，通过 URL 或 Base64 编码实现多模态数据传输，适配了图文问答、图像诊断等复杂场景；动态模型选择基于任务复杂度（如对话轮数）自动切换基础模型与高级模型，在保障效果的同时优化了算力成本；多模态模型集成则通过工具链封装，实现了文生图、语音识别等跨模态功能的整合；结构化输出通过 Pydantic 模型与 JSON Schema 两种方式，将 LLM 非结构化输出转为强类型数据，大幅降低了下游系统的解析成本，提升了数据可用性与一致性。

(4) 表达式语言（LCEL）作为智能体组件化开发的核心语法，为复杂调用流程的构建提供了极简方案。通过管道符 | 实现提示词、模型、工具、解析器等组件的串联，核心依赖于 Runnable 组件的统一接口（`invoke()/stream()/batch()`）。本章详细介绍了基础 Runnable 组件（提示词模板、模型、解析器）与自定义 Runnable 组件（`RunnableLambda`、`RunnableParallel` 等）的用法，结合带工具、带记忆、并行/分支等实战场景，展现了 LCEL 在简化开发流程、提升组件复用性、增强调用链灵活性方面的核心价值。

整体而言，本章内容层层递进，从交互体验增强（流式处理）、功能扩展机制（中间件）、核心能力升级（高级智能体）到开发效率优化（表达式语言），全面覆盖了智能体高级开发的关键技术点。通过大量农业场景实战案例（如病虫害诊断、农业图像生成、农田问题咨询），将理论与实践深度结合，不仅帮助开发者理解技术原理，更能直接应用于实际项目开发中。掌握本章内容后，开发者可灵活应对多场景、高复杂度的智能体开发需求，构建出更具实用性、稳定性与用户体验的 AI 应用。

课后练习题

一、选择题

- 1、LangChain 中实现异步流式输出，逐块返回生成内容的方法是（）
A. invoke () B. stream () C. astream () D. batch ()
- 2、流式传输 LLM Tokens 时，用于表示模型逐段生成内容片段的对象是（）
A. AIMessage B. AIMessageChunk C. ToolMessage D. HumanMessage
- 3、要实现 Agent 执行步骤的进度跟踪，需设置 stream/astream 方法的 stream_mode 参数为（）
A. "messages" B. "custom" C. "updates" D. "logs"
- 4、LangChain 中间件的本质是（），可在智能体关键执行节点插入自定义逻辑。
A. 预定义的工具集 B. 可插拔的执行钩子
C. 结构化的输出解析器 D. 多模态的消息载体
- 5、用于在模型调用前触发，实现参数校验、日志记录的生命周期钩子装饰器是（）
A. @before_model B. @after_model
C. @wrap_model_call D. @dynamic_prompt
- 6、要根据运行时上下文动态生成系统提示词，需使用的中间件装饰器是（）
A. @before_agent B. @dynamic_prompt
C. @after_tool D. @wrap_tool_call
- 7、LangChain 中实现结构化输出的核心工具，同时支持数据验证与类型约束的是（）
A. JSON Schema B. Pydantic 模型
C. StrOutputParser D. JsonOutputParser
- 8、LangChain 表达式语言（LCEL）中，用于串联多个 Runnable 组件的核心符号是（）
A. & B. | C. && D. ||
- 9、所有能被 LCEL 管道符串联的组件，本质上都实现了（）抽象接口。
A. BaseModel B. Runnable C. AgentState D. BaseChatModel
- 10、用于将普通函数封装为 Runnable 组件，适配 LCEL 管道语法的类是（）
A. RunnableSequence B. RunnableParallel
C. RunnableLambda D. RunnableBranch

二、简答题

- 1、简述 LangChain 中 invoke () 与 astream () 两种调用方法的核心区别，及流式处理的核心优势。
- 2、说明 LangChain 中间件的核心作用，并列举 3 个常用的生命周期钩子函数及各自的触发时机与典型用途。
- 3、简述 LangChain 实现结构化输出的两种方式（Pydantic 模型 / JSON 格式），并说明二者的适用场景。
- 4、什么是 LangChain 表达式语言（LCEL）？简述其核心语法及 Runnable 组件的核心特征。

三、综合应用题

基于 LangChain 框架构建一个农业多模态智能诊断体，需实现图像病虫害诊断、流式输出结果、Pydantic 结构化返回、LCEL 链式调用功能，回答以下问题并完成代码逻辑梳理：

- （1）梳理该智能体的核心技术栈，说明流式处理、多模态消息、结构化输出、LCEL 在本场景中的具体应用价值。

(2) 定义一个 Pydantic 模型 `AgriDiseaseResult`，用于结构化返回病虫害诊断结果，要求包含作物名称、病虫害名称、风险等级（低 / 中 / 高）、防治建议（不超过 100 字）四个字段，并添加字段约束与描述。

(3) 编写核心 LCEL 调用链代码，实现多模态输入（图像 URL）→ 提示词模板 → 多模态 LLM → Pydantic 解析器的完整链式调用，要求代码结构完整、注释清晰，使用硅基流动多模态模型 `Qwen/Qwen3.5-4B`。

教材未出版，禁止传播，版权所有：柯善斌

教材未出版，禁止传播，版权所有：柯善斌

本章小结

本章围绕 LangChain 智能体完整开发体系展开，搭建从理论原理到工程实战的完整知识框架，系统拆解 AI 智能体实现自主任务执行的全套技术方案。

核心理论层面：明确智能体区别于普通 LLM 的四大特征（自主性、交互性、目标导向、适应性），梳理六大智能体分类与五层分层架构，阐释感知 - 记忆 - 推理 - 规划 - 执行标准闭环运行逻辑，厘清 Agent、LLM、Tools、Memory、OutputParser 五大核心组件协同关系。

基础开发能力：讲解极简三角智能体底层逻辑，区分提示词与结构化消息从属关系，详解 System/Human/AI/Tool 四类消息优先级与使用场景；提供 ChatOpenAI、init_chat_model 两套模型标准化初始化方案，说明 Token 计费与上下文底层原理。

工程化高级能力：

工具：基于 @tool 装饰器规范自定义业务工具，实现外部 API、本地函数接入，解决大模型幻觉、知识滞后问题；

记忆：区分 InMemory 短期内存记忆、Redis 持久化长期记忆，依靠 thread_id 实现会话隔离与多轮连贯对话；

结构化输出：支持 JSON 轻量化输出、Pydantic 强类型校验输出，统一接口返回数据格式，降低下游解析成本；

流式交互：提供 messages、custom、updates 三种流式模式，分别实现打字机实时输出、工具执行日志推送、Agent 分步流程监控，配套同步 stream、异步 astream 适配前后端实时聊天场景。

整章搭配气象、农业、文本处理多场景实战代码，统一模型、环境、代码规范，学完可独立开发具备自主工具调用、多轮记忆、标准化输出、实时流式交互的商用智能体，支撑问答、诊断、资讯类 AI 应用落地。

课后练习题

一、选择题

1、智能体突破传统 LLM 的核心优势是实现了（ ）的升级，从“文本生成”变为“任务执行”。

- A. 感知 - 思考 - 行动 - 观察闭环
- B. 单一文本处理能力
- C. 更大的参数规模
- D. 更快的推理速度

2、以下不属于智能体核心特征的是（ ）

- A. 自主性
- B. 单向传输性
- C. 目标导向性
- D. 适应性

3、无记忆、仅基于当前环境状态响应的智能体类型是（ ）

- A. 目标导向智能体
- B. 反应式智能体
- C. 学习型智能体
- D. 基于模型的智能体

3、LangChain Agent 中负责决策者角色，规划并选择工具的核心组件是（ ）

- A. Tools
- B. Memory
- C. Agent
- D. LLM

4、LangChain 中承载模型输入输出、表示对话状态基本单位的是（ ）

- A. 提示词
- B. 消息
- C. 模板
- D. 工具

5、用于预设模型行为、定义角色和响应指南的消息类型是（ ）

- A. HumanMessage
- B. SystemMessage
- C. AIMessage
- D. ToolMessage

6、LangChain v1.0 中新增的通用模型初始化函数，可统一各类聊天模型初始化入口的

是 ()

- A. ChatOpenAI B. init_chat_model
- C. BaseChatModel D. ChatPromptTemplate

7、LangChain 中创建自定义工具最便捷的方式是使用 () 装饰器注解函数。

- A. @agent B. @tool C. @memory D. @prompt

8、LangChain 中实现短期记忆的核心类，将对话历史保存在内存中的是 ()

- A. RedisSaver B. InMemorySaver
- C. VectorStoreRetrieverMemory D. BaseStore

8、为 LangChain Agent 添加记忆能力时，在调用 invoke 方法时必须传入的可配置参数是 ()

- A. model_name B. system_prompt C. thread_id D. tools

二、简答题

- 1、简述智能体与传统大语言模型 (LLM) 的核心区别，及智能体的四大核心特征。
- 2、说明 LangChain 中消息与提示词的关系，并列举 3 种核心消息类型及各自作用。
- 3、简述 LangChain 工具的核心使用流程，并说明定义自定义工具时的关键注意事项。
- 4、对比智能体短期记忆与长期记忆的区别，分别说明二者在 LangChain 中的实现方式及核心类。

三、综合应用题

1、基于 LangChain 框架构建一个智能快递查询智能体，需实现实时快递信息查询、多轮对话记忆功能，回答以下问题并完成代码逻辑梳理：

(1) 梳理该智能体的核心组成模块 (结合 LangChain Agent 五大组件)，并说明每个模块在本场景中的具体作用。

(2) 定义一个自定义工具 `get_express_info`，实现根据快递单号查询快递信息的功能，要求包含工具描述、参数说明、基础异常处理。

(3) 编写核心代码逻辑，实现该智能体的完整构建：包括模型初始化 (使用硅基流动 Qwen3-8B)、工具注册、短期记忆配置、智能体创建及多轮调用，要求代码结构完整、注释清晰。